

LAM TVRMS Guide – Explained in Simple English

Oct 7, 2026 · @Syed Rahat

1. Start here – the PPT, the PDF, and how they connect

The PPT (ABC_REQUEST.pptx) is a 10-slide proposal from HCLTech to LAM. The PDF (81 pages) is a developer's build guide written on top of those 10 slides. The PDF is not a copy of the PPT: it adds many build details, and every detail the PPT did not give is marked **(ASSUMPTION)**.

A simple way to see it: the PPT is like the plan an architect shows to a customer – "3 rooms, 1 kitchen, ready in 9 weeks, this price". The PDF is the builder's working manual – exact room sizes, where each pipe goes, and in which order to build. Where the customer gave no detail (for example the tile colour), the builder wrote a sensible guess and marked it "to confirm".

Which slide feeds which part of the PDF

Slide	Slide title	What the slide says (short)	Where the PDF uses it
1	Cover	HCLTech's Solution Approach – LAM TV Request Management System – September 2026	Cover page, README
2	Table of Contents	7 topics	Not used for building
3	Objectives & Scope	Today's problem, the goal, 13 in-scope items, 4 out-of-scope items	Doc 00, Doc 01, and every build chapter
4	Business Process Flow	6 phases, with user journey, key actions and the email sent in each phase	Doc 01 (phases), Doc 03 (statuses), Doc 06 (buttons), Doc 08 (emails)
5	Solution Architecture	Entra ID, Canvas app, Model-driven app, Power Automate, Power BI, Dataverse, SharePoint, Outlook	README, Doc 02 to Doc 09

Slide	Slide title	What the slide says (short)	Where the PDF uses it
6	Execution Approach	Plan for Success, then Design/Build/Test in 2 sprints, then Deploy, then Hypercare	Doc 10, Doc 11, Doc 12
7	Execution Timeline	9 weeks: W1-W2 plan, W3-W6 two sprints, W7 go-live, W8-W9 hypercare	Doc 12
8	Assumptions & Dependencies	LAM gives master data, licences, experts (SMEs) and environments; business users do UAT	Doc 00, Doc 02, Doc 10
9	Non-Functional Requirements	24x7, Microsoft Edge, files in SharePoint, 7-year retention, full audit, role-based access, scalable	Doc 02 (audit), Doc 04 (retention), Doc 05 (security)
10	Commercials	Fixed price USD 58,950, taxes extra	Only used to explain "change requests" - no developer work

Two kinds of content in the PDF

- **Facts from the PPT.** Example: the 6 phases, the 4 request types (New, Repeat, One-time, Recurring), the 7-year retention.
- **Guesses added by the guide, marked (ASSUMPTION).** Example: the 15 statuses, "reminder after 3 days", "escalate after 5 days", "auto-cancel after 30 days", the USD 50,000 limit for a second approval, the folder names 01 to 06, and about 200 users and 500 requests a year.

Example: slide 3 only says "reminders, escalations, automatic cancellation". It does not say after how many days. The guide chose 3, 5 and 30 days and stored them in a settings table, so the business can change them later without touching any code.

What is inside the PDF

- **Part A - 16 chapters:** a README, Doc 00 to Doc 13, and a short README for the sample data.
- **Part B - 10 sample data files (CSV):** fake test data for departments, locations, research institutions, cost centers, settings, email templates, requests, feasibility options, milestones and costs.

You will see file names like docs/03-Dataverse-Data-Model.md. The PDF was printed from a folder of text files (a Git repository whose README is titled "RADIANCE"). The file names only show where each chapter came from; you do not need them to build.

How to read this explanation

Each section below follows the PDF in the same order. Each one gives the idea first, then the steps, then an example with the sample data. The last section (22) lists a few points in the guide that you should correct or check before you build.

The same sample people appear everywhere in the guide:

Person	Role in the story
Priya Sharma	Requester – a process engineer who needs a test
David Chen	APPG reviewer – checks and manages the request
Maria Lopez	Cost Center Owner of CC-4100 – approves the money
James Wilson	Maria's manager – second approver for amounts above USD 50,000
Tom Baker	Program manager – a read-only "stakeholder"
John Miller	A second requester (Metrology), used to test that people cannot see each other's requests

The main sample request is **TV-2026-00015 – "Plasma etch uniformity test on 300mm wafers"**. Section 20 follows it from the first click to the final report.

2. README – the project, the tools, and the reading order

The README says it in one line: LAM handles Test Vehicle requests today by email, PowerPoint, Excel and loose files, and this project replaces that with one Power Platform app. In that app every request moves through 6 phases, its files sit in one SharePoint folder, emails go out automatically, and managers see reports in Power BI.

What is a "Test Vehicle" (TV)?

LAM (Lam Research) builds machines that are used to make computer chips. Its engineers often need a test done – for example a test run on silicon wafers, a prototype, or a lab test at a university or a vendor. Each such test is a "Test Vehicle", and asking for one is a "TV request".

Example: Priya needs to test a new plasma etch recipe on 25 wafers at an outside lab. That is one TV request. The system gives it a number, TV-2026-00015.

The 8 tools, in plain words (from slide 5)

Layer	Tool	Plain meaning	In this project
Login and security	Microsoft Entra ID	The company login (earlier called Azure Active Directory)	Users sign in with their LAM account – no new password
Screens for normal users	Power Apps Canvas App	An app where you design every screen yourself, like drawing on a blank canvas	Create, review, assess, fund, execute and close requests
Screens for admins	Power Apps Model-Driven App	An app that Power Apps builds for you from the tables (lists and forms)	Master data, settings, email logs, error logs, audit
Automation	Power Automate	"Robots" that run in the background	Emails, approvals, reminders, folder creation, PDF creation
Data	Dataverse	Microsoft's database inside Power Platform	All request data, security roles, audit history
Documents	SharePoint Online library	The file store	One folder ("workspace") per TVID
Emails	Outlook (Office 365)	The mail system	Sends every notification
Reports	Power BI	Dashboards and charts	Status, cycle time, aging, cost, utilization

One click, all 8 tools – an example

When Priya clicks Submit on TV-2026-00015, this happens:

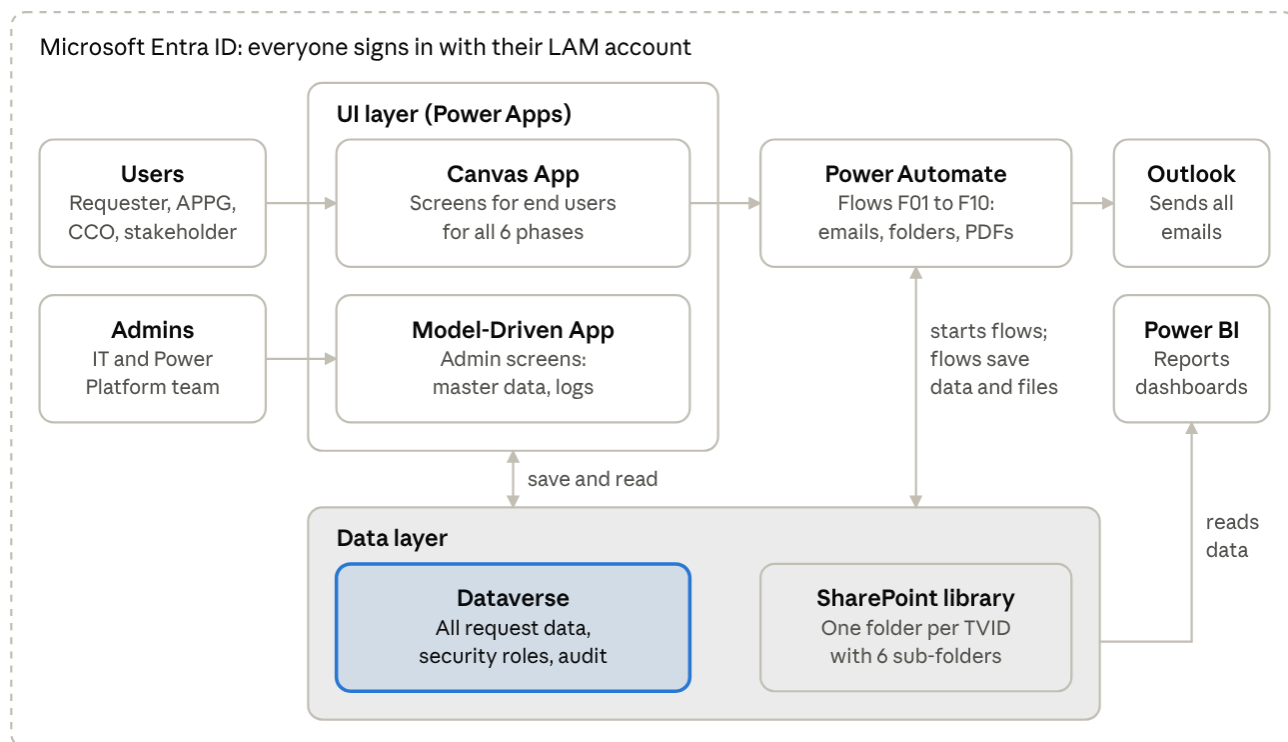
1. Entra ID already knows the user is Priya, because she signed in with her LAM account.
2. The Canvas app saves the request in Dataverse and sets the status to "Submitted".
3. A Power Automate flow sees the new status and creates the SharePoint folder TV-2026-00015 with 6 sub-folders.
4. The same flow sends emails through Outlook: a thank-you email to Priya and a "new request" email to the APPG team.

5. At the next refresh, Power BI counts the request under "Open requests".

The drawing below shows the same parts and how they talk to each other.

Both apps work on one data layer; flows send emails and make folders

From slide 5 (Solution Architecture) of ABC_REQUEST.pptx



TVRMS architecture · 8 tools in 2 layers

Everyone signs in through Entra ID. Both apps save and read data in Dataverse; a status change there starts a flow, and the flows save files in SharePoint and send emails through Outlook. Power BI only reads the data.

Why the chapters are in this order

The guide builds from the bottom up, like a house: ground first, roof last. You cannot build a screen before its table exists, and a flow cannot send an email about data that is not there.

1. Doc 00 and Doc 01 – understand the project and its gaps.
2. Doc 02 – prepare the ground: licences, environments, the solution, the SharePoint site.
3. Doc 03 – the data (tables and links).
4. Doc 04 and Doc 05 – the file store and the security.
5. Doc 06 and Doc 07 – the two apps (screens).
6. Doc 08 – the automation (flows).
7. Doc 09 – the reports.

8. Doc 10 to Doc 12 – testing, go-live and the 9-week plan.
9. Doc 13 – one full example story that ties everything together.

The README's quick answer

"Is the PPT enough to build the project?" – **No, not fully.** It is enough to start, but not enough to finish. Section 3 explains why, and what to ask.

3. Doc 00 – Is the PPT enough to build the project?

The guide's answer: the PPT is enough to **start** the project but not enough to **finish** it. It tells you what to build, with which tools, in how many weeks and at what price. It does not give the small details a developer needs, such as every form field, every status rule, or the text of every email.

This is normal for a proposal. Slides 6 and 7 keep Weeks 1–2 ("Plan for Success") for exactly this work: collect the missing details and write a **Solution Blueprint**.

What is a Solution Blueprint? A design document that lists every field, status, screen, flow, email, report number (KPI) and security rule. The business reads it and signs it. Only after that signature does the building (Sprint 1) start.

What the PPT gives you clearly (you can build these directly)

1. The 6 phases: Request Initiation, APPG Review, Internal Alignment, Funding Approval, Execution, Closure.
2. The 4 request types: New, Repeat, One-time, Recurring.
3. On submit, the system must check mandatory fields and the technical document.
4. APPG can review, assign an owner, comment, ask for more information, or reject.
5. Feasibility means comparing several research institutions or vendors on capability, cost, timeline, constraints and a recommendation.
6. Two internal alignment steps: 1st = get quotes and estimates; 2nd = confirm the chosen option and raise the funding request.
7. Funding: find the Cost Center Owner, send the approval, save the decision, comments and history.
8. Execution: milestones, planned and actual dates, progress, risks, costs, comments, expected completion date.
9. Closure: deliverables, outcomes, lessons learned, costs, reuse recommendation.
10. SharePoint: an automatic TVID folder with fixed sub-folders, versioning, permissions and 7-year retention.

11. Automation: notifications, reminders, escalations, status updates, automatic cancellation, approvals, audit log, PDF.
12. Roles: Requester, APPG user, Cost Center Owner, agreed stakeholders, system admin.
13. Power BI: status, cycle time and aging, cost, utilization and performance, exceptions, executive view.
14. Out of scope: connecting to other systems, moving old data, user training, change management.

The 20 gaps and what the guide assumed

Each gap is a detail the PPT does not give. The guide filled each one with a sensible guess so you can build now and adjust later.

Gap	What is missing	Why it matters	The guide's guess (ASSUMPTION)
G1	The full list of form fields	You cannot build a table or a form without it	About 25 fields; Doc 03 ends with 32 columns (some are filled automatically)
G2	What "APPG" stands for	It is the main reviewing team	A central planning and governance team
G3	The exact statuses and allowed moves	They drive the whole workflow	15 statuses (section 7)
G4	Which documents are "mandatory technical documents"	Needed for the submit check	At least 1 file of type "Technical Specification"
G5	Days before reminder, escalation, auto-cancel	Needed for the daily flow	3, 5 and 30 days, kept in a Settings table
G6	Funding approval rules	Needed for the approval flow	1 approver (the Cost Center Owner); above USD 50,000 also the owner's manager
G7	How to find the Cost Center Owner	The approval must go to the right person	Read the Owner column of the Cost Center table
G8	Who the "agreed stakeholders" are	Security	Read-only people added to each request (an "access team")

Gap	What is missing	Why it matters	The guide's guess (ASSUMPTION)
G9	SharePoint folder names	The folder flow needs them	01-Request to 06-Final Report & Closure
G10	Email subjects and texts	Notifications	Kept in an Email Template table
G11	What goes into the PDF	The PDF flow	Summary, feasibility, approval and closure
G12	Rules for Recurring requests	Data and flows	Store the frequency; no automatic copies
G13	Exact report numbers (KPIs)	Power BI	Defined in Doc 09
G14	Number of users and requests	Licences and speed	About 200 users and 500 requests a year
G15	Master data (users, departments, cost centers, locations, vendors)	Dropdowns and routing	LAM must provide it (slide 8); sample CSVs until then
G16	Environments and licences	Dataverse needs Power Apps Premium	Dev, Test, Prod; a premium licence for every app user
G17	Who does UAT and signs off	Slide 8 says business users	3 to 5 key users
G18	Logo and colours	Screen design	LAM logo, simple company colours
G19	Language, time zone, currency	Data formats	English, USD, LAM's local time zone
G20	Can a requester cancel their own request?	Status rules	Yes, while the status is Draft, Submitted or More Info Required

Two small points to watch inside these guesses:

- G5 says "3 **working** days", but the reminder flow later (F05, section 15) counts **calendar** days. Question 11 below asks the business to decide.
- G12 talks about a "Create next occurrence" button. In the later chapters this becomes an email (template NEXT_OCCURRENCE) that tells the requester to make a new "Repeat" request. Decide which one you want.

The 21 questions to send to your manager or the customer

The guide gives a ready email titled "LAM TVRMS - Clarification questions for Solution Blueprint". Send it in Week 1 and ask for answers before the end of that week.

#	Question (simple words)	Why we ask	Closes gap
1	What does APPG stand for? Who are the APPG users?	To create the APPG group and role	G2
2	What is the full list of statuses, and who may move a request between them?	To build the status rules	G3
3	For a Repeat request, should the app copy data from the old TVID?	To build the copy feature	G1
4	For Recurring requests, which frequencies are allowed? Should the system create the next request by itself?	To build Recurring correctly	G12
5	What exactly is the difference between the 1st and 2nd Internal Alignment? Who takes part?	To design those two steps	G3
6	How many options are usually compared? Is there a scoring method?	To design the feasibility screen	G1
7	One approver or several? Any amount limits?	To build the approval flow	G6
8	Can a request be put On Hold? Who can cancel?	Status rules	G3, G20
9	After how many days do we remind? After how many days do we escalate, and to whom?	The daily flow	G5
10	After how many days without an answer do we auto-cancel?	The daily flow	G5
11	Working days or calendar days?	Day counting	G5
12	Please share today's request template (PPT or Excel).	To map every field	G1
13	Which technical documents are mandatory at submit?	The submit check	G4

#	Question (simple words)	Why we ask	Closes gap
14	Please confirm the folder names for each TVID.	The folder flow	G9
15	Please share the master data.	Dropdowns and routing	G15
16	Please share or approve the email texts.	Notifications	G10
17	What should the PDF contain?	The PDF flow	G11
18	Which report numbers matter most? Any targets, for example "cycle time under 30 days"?	Power BI	G13
19	Which tenant and environments do we use? Who gives access?	Setup	G16
20	Do all users have Power Apps Premium and Power BI Pro licences?	Setup	G16
21	How many users and requests per year?	Licences, speed	G14

The guide's 4 recommendations

1. **Do not wait for all answers.** Create the environments and the data model with the guesses now; they are easy to change.
2. **Hold 2 or 3 workshops in Week 1:** one on the process, one on data and documents, one on reports.
3. **Write the Solution Blueprint in Week 2 and get it signed.** Slide 6 says "Report submitted for final approval". Only then start Sprint 1.
4. **After sign-off, every new idea is a Change Request.** The price is fixed (slide 10), so new scope can change cost and dates.

Example of a Change Request: in Week 5 the business asks, "Please also send SMS alerts." SMS was never in the signed scope. So you raise a Change Request; LAM and HCLTech agree on extra cost and time before you build it.

4. Doc 01 – Project understanding

This chapter explains the business story: 5 kinds of people, 6 phases, and the words you will meet everywhere. Read it before you build anything.

The problem today

Today an engineer asks for a Test Vehicle like this:

1. She sends an email to the APPG team.
2. She attaches a PowerPoint with the idea and an Excel sheet with the costs.
3. Vendor quotes come back by email.
4. Funding is approved by email.
5. Nobody knows the current status without asking someone.
6. Files sit in mailboxes and personal folders.

The result: delays, missing information, no history and no reports.

The solution in one picture of words

One app. Every request gets a unique number called the **TVID** – for example TV-2026-00015. The request travels through 6 phases. All its files sit in one SharePoint folder named after the TVID. Every step sends an email by itself. Managers see everything in Power BI.

The 5 roles (who uses the system)

Role	Who is this?	What can they do?	Sample person
Requester	An engineer who needs a test	Create, submit and update own request, answer questions, cancel own early request	Priya Sharma (Process Engineering)
APPG user (reviewer)	The central team that reviews requests (exact meaning to be confirmed)	Review, assign an owner, ask for more info, reject, run feasibility and alignment, track execution, close	David Chen
Cost Center Owner (CCO)	The manager who owns the budget	Approve or reject the money	Maria Lopez (CC-4100)
Agreed stakeholder	People who must be able to see a request	Read only	Tom Baker (Program Manager)
System Administrator	IT / Power Platform admin	Master data, settings, logs, security	Admin Team

The 6 phases (from slide 4)

A request normally moves through the 6 phases in order. There are only 2 early exits: APPG can **reject** it in phase 2, and the Cost Center Owner can **reject the funding** in phase 4. Also, in phase 2 APPG can send it back to the requester for more information, and the requester answers and resubmits.

Phase	What happens (user journey)	Main actions	Email sent	Example with TV-2026-00015
1. Request Initiation	Requester creates and submits	Enter details and justification, upload files, submit; the system checks mandatory fields	Thank-you (acknowledgement) to requester	Priya fills the form, attaches Etch_Test_Spec_v1.pdf, submits on 12-Oct-2026
2. APPG Review	APPG checks the request and does a first feasibility check	Review, ask for more info or reject, send to internal alignment	Review started / more info needed / rejected	David asks for the wafer map; Priya adds it; David moves it on
3. Internal Alignment	Quotes, funding review, confirmation	1st: collect options, quotes, estimates, clarify terms. 2nd: confirm the chosen option and raise the funding request	Alignment update, funding request	3 labs quote; Stanford (USD 39,500) is chosen
4. Funding Approval	The budget owner decides	Approve: execution can start. Reject: requester is told the reason	Funding approved / rejected	Maria approves USD 39,500 on 3-Nov

Phase	What happens (user journey)	Main actions	Email sent	Example with TV-2026-00015
5. Execution	The work is done and tracked	Do the work, track progress, costs and issues	Status changes, reminders, escalations, exceptions	4 milestones, 1 issue, costs USD 37,800
6. Closure	Request closed and results stored	Upload final report, closure email, audit trail	Completed, report available	Closed on 5-Mar-2027 with a summary PDF

Glossary – words you will see everywhere

Word	Simple meaning	Example
TV	Test Vehicle – the test being requested	A plasma etch test on 25 wafers
TVID	The unique request number	TV-2026-00015
APPG	The reviewing team (full name not known yet)	David Chen's team
RI	Research Institution – an outside lab, university or vendor that can do the test	Stanford Nanofab Lab
Feasibility	"Can it be done, by whom, at what cost, in what time?"	3 options compared
CCO	Cost Center Owner – approves the money	Maria Lopez
Dataverse	Microsoft's database inside Power Platform	Holds the TV Request table
Canvas App	A Power App where you design each screen yourself	The app Priya uses
Model-Driven App	A Power App generated from the Dataverse tables	The admin app
Flow	One automation in Power Automate	F01 creates the folders

Word	Simple meaning	Example
Solution	A "package" that holds all your parts, so you can move them from Dev to Test to Prod	The solution named TVRMS
UAT	User Acceptance Testing - business users test before go-live	3 to 5 key users in Week 6-7
Hypercare	Extra support for about 2 weeks after go-live	Weeks 8-9
RBAC	Role-Based Access Control - what you can see and do depends on your role	Priya sees only her own requests

The 4 request types (from slide 3)

Type	Meaning	Example	What the app does differently
New	A first-time request for a new test	"New plasma etch test on 300mm wafers"	Nothing special
Repeat	The same as an earlier request, maybe with a small change	"Repeat TV-2026-00003 with new chamber material"	Asks for the Previous Request and copies its details
One-time	A test done only once	"One-time thermal stress test"	Nothing special
Recurring	A test that happens again and again	"Quarterly contamination check at a university lab"	Asks for the frequency (Monthly, Quarterly, Half-yearly, Yearly)

Out of scope - do not build these

Not in scope	What it means for you
Integration with external systems (for example SAP or ERP)	No live link to finance or other systems. Costs are typed into the app by hand.
Migration of existing data	Old requests stay in email and Excel. The app starts empty, except for master data.

Not in scope	What it means for you
Training for business users	LAM trains its own users. You still run the short UAT kick-off and the support handover.
Change management	LAM sends the go-live news and manages the people side of the change.

If someone asks for one of these during the project, it is a Change Request (section 3).

5. Doc 02 – Environment setup (do this first)

Before you build any table or screen, you prepare the ground in 9 steps: licences, 3 environments, auditing, a publisher and solution, a SharePoint site, environment variables, connection references, security groups, and laptop tools. If you skip this, you will rebuild things later.

Step 1 – Check the licences

Idea. Dataverse is a "premium" feature. Anyone who opens an app that uses Dataverse needs a premium Power Apps licence. Slide 8 says LAM provides all licences from Day 1.

Who	Licence	Why
Every app user (requesters, APPG, CCOs, stakeholders)	Power Apps Premium (per user) or Per App	The app uses Dataverse, a premium connector
Every report viewer	Power BI Pro (or Premium capacity)	To view shared dashboards
Developers	Power Apps Premium + Power BI Pro (or the free Developer Plan for Dev)	To build and test
Flow owner (a service account)	Power Automate Premium (or covered by the Power Apps licence when the flow serves the app)	Flows use Dataverse, SharePoint and Outlook

Example: if Priya has only a normal Microsoft 365 licence, the app shows a licence error when she opens it.

Service account. Use one non-personal account, for example svc-tvrms@lam.com, as the owner of all flows and connections. Then all emails come from one mailbox, and flows do not stop when a developer leaves the company.

Step 2 – Create 3 environments

Idea. An environment is a separate room with its own Dataverse database. You build in one room, test in a second, and real users work in a third. A mistake in the build room never touches real data.

Name	Type	Dataverse	Used for
LAM-TVRMS-DEV	Sandbox	Yes	Developers build here
LAM-TVRMS-TEST	Sandbox	Yes	Testing and UAT
LAM-TVRMS-PROD	Production	Yes	Real users

Steps:

1. Open <https://admin.powerplatform.microsoft.com> and go to **Environments** → **+ New**.
2. Type the name, choose the type, and switch **Add a Dataverse data store** to Yes.
3. Set Language = English and Currency = USD.
4. Set the **Security group** to an Entra ID group such as SG-TVRMS-Users. Now only members of that group can enter the environment.

A "sandbox" can be reset or copied for testing; a "production" environment is for live business use.

Step 3 – Turn on auditing

Idea. Auditing makes Dataverse record who changed what and when, with the old and the new value. Slide 9 asks that all actions are auditable for 7 years.

Steps: Power Platform admin center → your environment → **Settings** → **Audit and logs** → **Audit settings**.

1. Tick **Start auditing**.
2. Tick **Log access** (records when people open records).
3. Retain logs: **Custom** → **2555 days**. $7 \text{ years} \times 365 \text{ days} = 2555$. Confirm this number with LAM's compliance team.

Example of what auditing later shows: "Request Status changed from Submitted to Under APPG Review by David Chen, 13-Oct 09:12."

Note: auditing must be on at two levels – the environment (this step) and each table (Doc 03 turns it on for the TV Request table). Turn it on for every table you want tracked.

Step 4 – Create a publisher and a solution (in DEV)

Idea. A **solution** is a box that holds everything you build – tables, apps, flows, roles, variables. You move the box from DEV to TEST to PROD. A **publisher** is the owner name on the box, and it gives a short **prefix** that starts every name you create.

Steps:

1. Go to <https://make.powerapps.com> and pick the environment LAM-TVRMS-DEV.
2. Click **Solutions** → **New solution**.
3. Click **New publisher** and fill: Display name = LAM TVRMS, Name = lamtvrms, Prefix = tvr.
4. Fill the solution: Display name = TV Request Management System, Name = TVRMS, Version = 1.0.0.0.

Because the prefix is tvr, every table and column starts with tvr_. Example: the TV Request table becomes tvr_tvrequest.

The version has 4 numbers: major.minor.build.revision. You raise it before every export, for example 1.0.0.5 → 1.0.1.0.

Golden rule: create everything inside this solution, never in the "Default Solution". Things outside your box do not move to TEST and PROD.

Step 5 – Create the SharePoint site

1. In SharePoint, click **Create site** → **Team site** (or Communication site).
2. Name it "TV Request Management". The address becomes something like <https://lam.sharepoint.com/sites/TVRMS>.
3. Create a document library named **TV Requests**. It will hold one folder per TVID.
4. Make one site per environment (ASSUMPTION): TVRMS-DEV, TVRMS-TEST and TVRMS (for production). This keeps test files away from real files.

Section 9 explains the folders, versioning, permissions and retention.

Step 6 – Create environment variables (inside the solution)

Idea. An environment variable is a setting inside the solution whose value is different in each environment. Flows read the variable instead of a fixed address, so you never edit a flow when you move it.

Display name	Schema name	Type	Example value in DEV
SharePoint Site URL	tvr_SPSiteUrl	Text	https://lam.sharepoint.com/sites/TVRMS-DEV
SharePoint Library	tvr_SPLibrary	Text	TV Requests
App URL	tvr_AppUrl	Text	https://apps.powerapps.com/play/e/.../a/...
APPG Team Email	tvr_APPGEmail	Text	appg-team@lam.com
Admin Email	tvr_AdminEmail	Text	tvrms-admin@lam.com

Example: tvr_SPSiteUrl is ".../TVRMS-DEV" in DEV, ".../TVRMS-TEST" in TEST and ".../TVRMS" in PROD. Flow F01 always reads tvr_SPSiteUrl, so it creates folders in the right site everywhere.

Steps: open the solution → **New** → **More** → **Environment variable** → fill name, type and value → Save.

Step 7 - Create connection references

Idea. A connection is a signed-in link to a service, for example "svc-tvrms's Outlook". A connection reference is a named plug point inside the solution. Each environment plugs a real connection into it once, and every flow uses the plug point.

Create these 6 (Solution → **New** → **More** → **Connection reference**):

Connection reference	Service	Used for
tvr_Dataverse	Microsoft Dataverse	Read and write request data
tvr_SharePoint	SharePoint	Folders and files
tvr_Outlook	Office 365 Outlook	Sending emails
tvr_Approvals	Approvals	Funding approval
tvr_Users	Office 365 Users	Finding a person's manager
tvr_OneDrive	OneDrive for Business	Turning HTML into PDF

Example: in TEST you plug the service account's Outlook connection into tvr_Outlook once, and all 10 flows can send email.

Step 8 – Ask IT for Entra ID security groups

Idea. A security group is a list of people that IT keeps. You give access to the group, not to single persons. When someone joins or leaves, IT only changes the group.

Group	Members (sample)	Linked to
SG-TVRMS-Users	All users	Entry to the environment
SG-TVRMS-Requesters	Engineers	Dataverse team with role "TVR Requester"
SG-TVRMS-APPG	APPG team	Role "TVR APPG Reviewer"
SG-TVRMS-CCOwners	Cost center owners	Role "TVR Cost Center Owner"
SG-TVRMS-Admins	IT admins	Role "TVR Admin"

Section 10 shows how these groups become Dataverse teams with roles.

Step 9 – Tools on your laptop (optional but useful)

- **Microsoft Edge** – slide 9 says the app must work in Edge.
- **Power BI Desktop** – free, from the Microsoft Store, to build the reports.
- **Power Platform CLI (pac)** – a command-line tool to export the solution into source control.
- **Visual Studio Code** with the "Power Platform Tools" extension.
- **Git** – to keep the exported solution in a repository (a folder named solution/).

Setup checklist

- ☐ Licences confirmed
- ☐ DEV, TEST and PROD environments created
- ☐ Auditing on, with 7-year retention
- ☐ Publisher tvr and solution TVRMS created
- ☐ SharePoint site and library created
- ☐ Environment variables and connection references created
- ☐ Security groups created

6. Doc 03 (part 1) – How the tables link together

The data model has 1 main table (TV Request), 6 master-data tables, 10 child tables, and 26 links between them. Every link is made by one "lookup" column, and the lookup always sits on the "many" side. If you understand this one rule, the whole chapter becomes easy.

First learn 4 simple words

Word	Simple meaning	Example in this project
Table	A list of records – like one sheet in Excel	TV Request, Department
Row (record)	One line in a table	The request TV-2026-00015
Lookup column	A column whose value is one row of another table – like picking from a dropdown	On the request, the Department column holds "Process Engineering"
Relationship (link)	The link that a lookup column creates between two tables	TV Request → Department

How to read the pictures in the PDF

- A short bar with **1** at the end of a line = the **one** side (the parent).
- A crow's foot (three short lines) with **N** = the **many** side (the child): zero, one or many rows.
- The arrow → means "points to". TV Request → Department means "a request points to one department". It always goes from the table that holds the lookup (N) to the one table (1).
- Colours in the PDF: blue = master data, purple = TV Request, green = child tables, grey dashed = User (people).
- A dashed loop under TV Request = the "self link" for Repeat requests.

Read every line as a sentence. "Department 1 — N TV Request" means: one department can have many requests, but each request belongs to only one department.

Golden rule: the lookup column is stored on the N side (the child). So the Department column is created on the TV Request table – not on the Department table.

The 3 pictures in the PDF

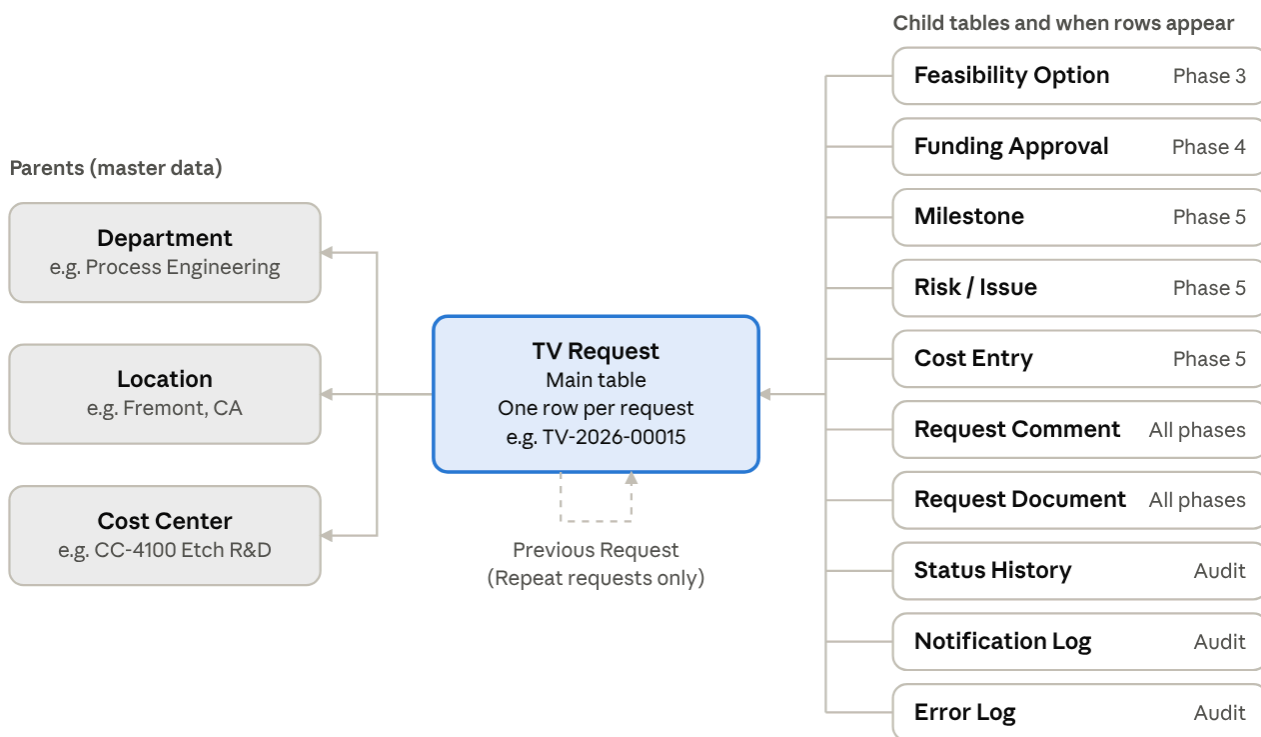
1. **Picture 1** – TV Request in the middle, its 3 master-data parents on the left (Department, Location, Cost Center), its 10 child tables on the right, and the self link. 14 links.
2. **Picture 2** – the other 12 links: 4 table links plus 8 "person" columns that point to the built-in User table.
3. **Picture 3** – the same links filled with real sample data for TV-2026-00015.

Three tables have no links at all: App Setting, Email Template and the optional Status Transition table. Flows read them by a key or a code, for example "ReminderAfterDays" or "REQ_SUBMITTED".

The drawing below shows Picture 1 in a simpler form.

Every child table points to TV Request; the request points to 3 parents

Simplified Picture 1 of Doc 03. Arrow = points to; the lookup column sits where the arrow starts.



TV Request and its links · 3 parents, 10 children, 1 self link

Read it from the right: each child row (for example a Milestone) points to one request, and each request points to one Department, one Location and one Cost Center. The other 12 links (person columns and the 4 extra table links) are in groups C and D below.

All 26 links, in 4 groups

A. Links from master data (4 links)

#	One (1)	Many (N)	Lookup column (on the N table)	Meaning	Example
1	Department	TV Request	Department	Each request belongs to one department	TV-2026-00015 → Process Engineering
2	Location	TV Request	Location	Where the request comes from	TV-2026-00015 → Fremont, CA
3	Cost Center	TV Request	Cost Center	The budget it is charged to; its owner approves	TV-2026-00015 → CC-4100 Etch R&D

#	One (1)	Many (N)	Lookup column (on the N table)	Meaning	Example
4	Department	Cost Center	Department	Each cost center belongs to one department	CC-4100 → Process Engineering

B. One request and its 10 child tables (10 links). Each child table has a lookup column named "TV Request". The last column shows what TV-2026-00015 has at the end of the story (closed on 5-Mar-2027).

#	Child table (N)	What one row means	TV-2026-00015 has
5	Feasibility Option	One possible way to do the test (lab, cost, time)	3 rows: A Stanford 39,500 · B Fraunhofer 35,000 · C Internal Lab 22,000
6	Funding Approval	One approval decision (one per approval level)	1 row: Maria Lopez - Approved - 39,500
7	Milestone	One step of the work, with planned and actual dates	4 rows: wafer preparation, etch runs, CD-SEM, final report
8	Risk / Issue	One problem or risk during execution	1 row: lab tool down - High - Resolved
9	Cost Entry	One real cost (an invoice)	4 rows = 37,800 (the sample CSV shows an earlier moment: 2 rows = 23,300)
10	Request Comment	One comment or question	2 rows: David asks for the wafer map, Priya answers
11	Request Document	One file's details (the file itself is in SharePoint)	7 rows: spec, wafer map, quote, approval PDF, progress report, final report, summary PDF
12	Status History	One status move (who, when, from, to)	11 rows: Draft → Submitted ... → Closed
13	Notification Log	One email the system tried to send	One row per email, for example REQ_SUBMITTED to Priya
14	Error Log	One failed flow run	0 rows - no flow failed in this story

C. Other table links (4 links)

#	One (1)	Many (N)	Lookup column	Meaning	Example
15	Research Institution	Feasibility Option	Research Institution	Which lab made this offer; one lab can make offers for many requests	Option A → Stanford Nanofab Lab
16	Cost Center	Funding Approval	Cost Center	Which budget the approval is for	Maria's approval → CC-4100
17	TV Request (old)	TV Request (new)	Previous Request (tvr_parentrequest)	Self link: a Repeat request points to the request it repeats; empty for New requests	TV-2026-00016 → TV-2026-00003
18	Feasibility Option	TV Request	Selected Option (tvr_selectedoption)	The one option chosen in Internal Alignment 2	TV-2026-00015 → Option A - Stanford

Link 18 looks strange because the request already has 3 options as children (link 5). But it is a normal lookup: the request also "points to" the one option that won.

D. Person columns → User (8 links). The User table is built in (the people come from Entra ID). One person can appear on many rows.

#	Table (N side)	Person column	Who is it?	Example
19	Department	Department Head	Head of the department	Ravi Kumar
20	Cost Center	Cost Center Owner	Approves the money (flow F04)	Maria Lopez
21	Cost Center	Owner's Manager	Second approver above USD 50,000	James Wilson
22	TV Request	Requester	The person who asks for the test	Priya Sharma
23	TV Request	APPG Owner	The reviewer who handles it	David Chen
24	Funding Approval	Approver	The person who decided	Maria Lopez
25	Risk / Issue	Owner	The person who must fix it	David Chen
26	Status History	Changed By	The person who moved the status	David Chen

How to create one link in Dataverse (step by step)

Example: link TV Request to Department (link 1).

1. Go to make.powerapps.com → environment LAM-TVRMS-DEV → **Solutions** → **TVRMS** → **Tables**.
2. Open the table on the N side – here **TV Request** (golden rule).
3. Click **+ New** → **Column**. Display name = Department. Data type = **Lookup**. Related table = **Department**.
4. Click **Save**. Dataverse creates the one-to-many relationship (one Department, many TV Requests) by itself.
5. Check it: open the Department table → **Relationships**. You see a one-to-many relationship to TV Request.
6. Only for the 10 child-table links (group B): change the relationship behaviour to **Delete = Restrict** and **Assign/Share = Cascade All** (section 8 explains why). Keep the default for all other links.

Why keep the default for the others? With "Cascade All" on Department, re-assigning one Department row would also re-assign every request of that department. That would be a serious accident.

Repeat the same steps for every link in groups A to D. For person columns, the related table is **User**.

3 common mistakes

1. **Lookup on the wrong table** – a Request lookup created on Department. Then one department could hold only one request.
2. **A text column instead of a lookup** – typing "Process Engineering" by hand. Then there is no real link, no dropdown, and no filter in reports.
3. **A many-to-many relationship** – none of the 26 links needs it.

7. Doc 03 (part 2) – Master tables, the TV Request table, and the 15 statuses

This part lists every column of the 6 master-data tables and of the main TV Request table (32 columns), and the 15 statuses a request can have. All names start with tvr_ because of the publisher prefix.

To create a table: Solution → **New** → **Table** → **Table**, type the name, then add the columns.

Two words you will see in the tables below:

- **Primary column** – every Dataverse table has one “name” column. It is the text people see in dropdowns and lookups.
- **Text 100** – a text column that holds up to 100 characters.

The 6 master-data tables (LAM provides the data – slide 8)

Department (tvr_department)

Column	Type	Required	Sample
Name (primary)	Text 100	Yes	Process Engineering
Department Code	Text 20	Yes	PE
Department Head	Lookup → User	No	Ravi Kumar
Is Active	Yes/No	Yes	Yes

Cost Center (tvr_costcenter)

Column	Type	Required	Sample
Name (primary)	Text 100	Yes	CC-4100 Etch R&D
Cost Center Code	Text 20	Yes	4100
Cost Center Owner	Lookup → User	Yes	Maria Lopez
Owner's Manager (for escalation and high amounts)	Lookup → User	No	James Wilson
Department	Lookup → Department	No	Process Engineering
Is Active	Yes/No	Yes	Yes

The Owner column is very important: the funding flow sends the approval to this person.

Location (tvr_location) – Name (Fremont, CA), Country (USA), Is Active (Yes).

Research Institution / Vendor (tvr_researchinstitution)

Column	Type	Sample
Name	Text	Stanford Nanofab Lab

Column	Type	Sample
Type	Choice: University, Research Lab, Vendor, Internal Lab	University
Contact Name	Text	Dr. Alan Green
Contact Email	Email	agreen@stanford.example
Capabilities	Multiline text	Plasma etch, SEM, TEM
Is Active	Yes/No	Yes

App Setting (tvr_appsetting) – the business can change rules here without any code change.

Name (key)	Value	What it controls
ReminderAfterDays	3	Send a reminder if an action has waited more than 3 days
EscalationAfterDays	5	Email the manager if it has waited more than 5 days
AutoCancelAfterDays	30	Cancel a request that stays in "More Info Required" for 30 days
HighValueFundingLimit	50000	Above USD 50,000 a second approval is needed
RetentionYears	7	How long records are kept

Example: the business says "remind after 2 days". The admin changes ReminderAfterDays from 3 to 2. The next daily run of flow F05 uses 2. No developer is needed.

Email Template (tvr_emailtemplate)

Column	Type	Sample
Template Code (primary)	Text	REQ_SUBMITTED
Subject	Text	[TVRMS] {TVID} submitted - {Title}
Body (HTML)	Multiline	<pre><p>Dear {RequesterName},</p><p>Your request {TVID} has been submitted...</p></pre>
Is Active	Yes/No	Yes

The words in curly brackets are placeholders. The flow replaces {TVID}, {Title}, {RequesterName}, {Status}, {Link} and {Comments} with real values before it sends the email. So "{TVID} submitted" becomes "TV-2026-00015 submitted".

"Is Active" rule for all master data: never delete a master-data row. Set Is Active = No instead. The dropdowns in the app show only active rows, so old requests keep their links and history.

The main table – TV Request (tvr_tvrequest), 32 columns

On this table, turn on **Auditing** and **Track changes**, and choose Ownership = **User or team** (needed for the security in section 10).

The "When filled" column tells you who fills it: "Form" = the requester types it; "Auto" = the app or a flow fills it; a phase name = it is filled later in that phase.

#	Display name	Schema name	Type	When filled	Sample
1	TVID (primary)	tvr_tvvid	Autonumber TV- {DATETIMEUTC:yyyy}- {SEQNUM:5}	Auto	TV-2026-00015
2	Request Title	tvr_title	Text 200	Form (required)	Plasma etch uniformity test on 300mm wafers
3	Request Type	tvr_requesttype	Choice: New, Repeat, One-time, Recurring	Form (required)	New
4	Previous Request	tvr_parentrequest	Lookup → TV Request	Form, only if Repeat	TV-2026-00003
5	Recurrence Frequency	tvr_frequency	Choice: Monthly, Quarterly, Half-yearly, Yearly	Form, only if Recurring	Quarterly
6	Requester	tvr_requester	Lookup → User	Auto (the signed-in user)	Priya Sharma
7	Department	tvr_department	Lookup → Department	Form (required)	Process Engineering
8	Location	tvr_location	Lookup → Location	Form (required)	Fremont, CA
9	Cost Center	tvr_costcenter	Lookup → Cost Center	Form (required)	CC-4100 Etch R&D
10	Business Justification	tvr_justification	Multiline 4000	Form (required)	Customer X reports 4% non-uniformity...
11	Technical Description	tvr_technicaldescription	Multiline 4000	Form (required)	25 wafers, 3 recipes, measure CD...
12	Expected Outcome	tvr_expectedoutcome	Multiline	Form (required)	Uniformity < 2%
13	Priority	tvr_priority	Choice: Low, Medium, High, Critical	Form (required)	High
14	Required By Date	tvr_requiredbydate	Date only	Form (required)	2026-12-15
15	Estimated Budget (USD)	tvr_estimatedbudget	Currency	Form (required)	42,000
16	Request Status	tvr_status	Choice (15 values, below)	Auto	Submitted

#	Display name	Schema name	Type	When filled	Sample
17	Current Phase	tvr_phase	Choice: Initiation, APPG Review, Internal Alignment, Funding Approval, Execution, Closure	Auto	APPG Review
18	APPG Owner	tvr_appgowner	Lookup → User	APPG Review	David Chen
19	Selected Option	tvr_selectedoption	Lookup → Feasibility Option	Internal Alignment 2	Stanford Nanofab – Option A
20	Approved Amount (USD)	tvr_approvedamount	Currency	Funding Approval	39,500
21	Expected Completion Date	tvr_expectedcompletion	Date only	Execution	2027-02-28
22	Overall Progress %	tvr_progress	Whole number 0-100	Execution	60
23	Actual Total Cost (USD)	tvr_actualcost	Currency, Rollup of Cost Entry	Auto	37,800
24	SharePoint Folder URL	tvr_spfolderurl	URL	Auto (flow F01)	https://lam.sharepoint.com/sites/TVRMS/TVRequests/TV-2026-00015
25	Submitted On	tvr_submittedon	Date and time	Auto	2026-10-12 10:05
26	Status Changed On	tvr_statuschangedon	Date and time	Auto	Used for reminders and aging
27	Closed On	tvr_closedon	Date and time	Auto	2027-03-05
28	Final Outcome	tvr_finaloutcome	Multiline	Closure	Uniformity improved to 1.6%
29	Lessons Learned	tvr_lessonslearned	Multiline	Closure	Book lab 6 weeks early
30	Reuse Recommendation	tvr_reuserecommendation	Choice: Reuse as-is, Reuse with changes, Do not reuse	Closure	Reuse with changes
31	Rejection / Cancellation Reason	tvr_reason	Multiline	When rejected or cancelled	Duplicate of TV-2026-00011
32	Cycle Time (days)	tvr_cycletimedays	Formula: DiffInDays(Submitted On, Closed On)	Auto	144

The special columns, explained:

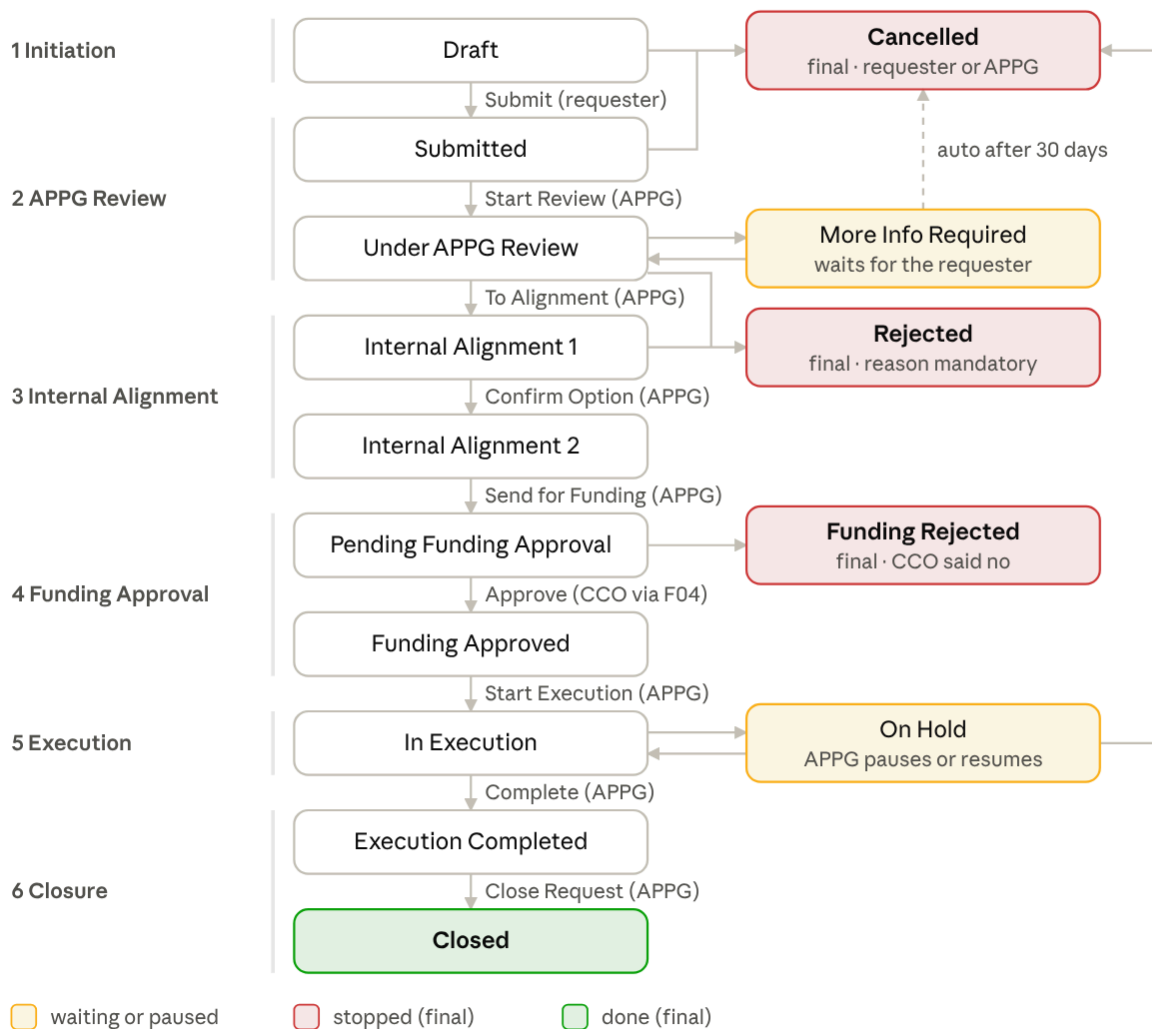
- **TVID (autonumber).** "TV-" + the 4-digit year + "-" + a 5-digit counter. The counter is one running number, so it does not restart at 00001 each January. If LAM wants a yearly restart, raise it in the workshop.
- **Request Status.** Do not name it just "Status". Dataverse already has system columns called Status (Active/Inactive) and Status Reason.
- **Actual Total Cost (rollup).** Dataverse adds up all Cost Entry amounts of the request. Example: 18,000 + 12,000 + 5,300 + 2,500 = 37,800.
- **Cycle Time.** The days from submit to close. Example: 12-Oct-2026 to 5-Mar-2027 = 144 days.
- **Status Changed On.** Updated on every status move. The reminder flow and the aging report use it to count waiting days.

The 15 statuses and the allowed moves (ASSUMPTION – confirm in the workshop)

The drawing below shows every status and every allowed move. Read it top to bottom: the middle column is the normal path; the boxes on the right are the side roads.

A request moves down one main path; it can stop at 4 final statuses

15 statuses from Doc 03 (ASSUMPTION). Words beside arrows = the button and who clicks it.



APPG can also cancel any open request. Back arrows: Resubmit (requester) and Resume (APPG).

TV Request statuses · 15 statuses, 4 final

Example: TV-2026-00015 went Draft → Submitted → Under APPG Review → More Info Required → back to Under APPG Review → Internal Alignment 1 → 2 → Pending Funding Approval → Funding Approved → In Execution → Execution Completed → Closed.

The rules behind the drawing:

- **4 statuses are final** (no move after them): Rejected, Funding Rejected, Closed, Cancelled.

- **The requester moves a request** only from Draft (submit) and from More Info Required (resubmit). Everything else is done by APPG, except the funding decision, which the Cost Center Owner makes through flow F04.
- **Cancel:** the requester can cancel in Draft, Submitted or More Info Required; APPG can cancel any open request; the system cancels a request that stays 30 days in More Info Required; an On Hold request can also be cancelled.
- **Status and phase go together.** Example: Internal Alignment 1 and 2 both belong to the phase Internal Alignment.

Tip from the guide: keep the "allowed next status" rules in one place – a collection in the Canvas app or a small table named `tvr_statustransition` – so they are easy to change after the workshop.

One small mismatch in the guide: the status table allows only "Internal Alignment 1 → Internal Alignment 2", but the Reject button in Doc 06 is also visible in Internal Alignment 1. Decide in the workshop whether APPG may reject at that stage; the drawing shows it as allowed.

8. Doc 03 (part 3) – The 10 child tables, safety settings, and loading sample data

Each child table stores one kind of detail for a request, and each has a lookup column named "TV Request" that points to its parent request. This part also explains two safety settings (Restrict and Cascade), four validation rules, and the order for loading sample data.

Three general points before the tables:

- **Created By / Created On** are filled by Dataverse on every row by itself. You do not create these columns.
- **From Status / To Status** (in Status History) must use the same global choice as Request Status, named "TVR Status" (section 11). Then the values always match.
- **Stage** (in Request Comment) uses the same global choice as Current Phase, named "TVR Phase".

1. Feasibility Option (`tvr_feasibilityoption`) – one way to do the test

Column	Type	Sample
Option Name (primary)	Text	Option A – Stanford Nanofab
TV Request	Lookup → TV Request	TV-2026-00015

Column	Type	Sample
Research Institution	Lookup → Research Institution	Stanford Nanofab Lab
Capability Score (1-5)	Whole number	5
Quoted Cost (USD)	Currency	39,500
Timeline (weeks)	Whole number	10
Constraints	Multiline	Lab available only from Nov
Recommendation	Choice: Recommended, Alternative, Not Recommended	Recommended
Is Selected	Yes/No	Yes
Quote Document URL	URL	Link to the quote file in SharePoint

Use case: David asks 3 labs for quotes and enters 3 rows. Only one row ends with Is Selected = Yes.

2. Funding Approval (tvr_fundingapproval) - the approval history

Column	Type	Sample
Name	Text (filled by the flow)	FA-TV-2026-00015-1
TV Request	Lookup	TV-2026-00015
Cost Center	Lookup	CC-4100
Approver	Lookup → User	Maria Lopez
Approval Level	Whole number	1
Requested Amount	Currency	39,500
Decision	Choice: Pending, Approved, Rejected	Approved
Comments	Multiline	Approved from Q4 budget
Requested On / Decided On	Date and time	2026-11-02 / 2026-11-03

The name is "FA-" + TVID + "-" + level. One row per approval level, so a USD 62,000 request gets 2 rows (level 1 Maria, level 2 James). This table is the "approval history" that slide 3 asks for.

3. Milestone (tvr_milestone) - one step of the work

Column	Type	Sample
Milestone Name	Text	Wafer preparation
TV Request	Lookup	TV-2026-00015
Sequence	Whole number	1
Planned Start / Planned End	Date	2026-11-10 / 2026-11-24
Actual Start / Actual End	Date	2026-11-12 / 2026-11-26
% Complete	Whole number	100
Status	Choice: Not Started, In Progress, Completed, Delayed	Completed

The "Sequence" number keeps milestones in the right order on the screen (1, 2, 3, 4).

4. Risk / Issue (tvr_riskissue) - one problem during execution

Column	Type	Sample
Title	Text	Lab tool down for maintenance
TV Request	Lookup	TV-2026-00015
Type	Choice: Risk, Issue, Exception	Issue
Severity	Choice: Low, Medium, High	High
Owner	Lookup → User	David Chen
Mitigation / Action	Multiline	Move run to 2nd lab
Status	Choice: Open, In Progress, Resolved	Resolved
Due Date	Date	2026-12-05

Simple meanings: a **risk** might happen ("the lab may be busy in December"); an **issue** has already happened ("the tool is down"); an **exception** is something outside the normal process. A row with Severity = High makes flow F02b send an alert email.

5. Cost Entry (tvr_costentry) – one real cost

Column	Type	Sample
Description	Text	Lab usage fee – Nov
TV Request	Lookup	TV-2026-00015
Category	Choice: Lab Fee, Material, Shipping, Labor, Other	Lab Fee
Amount (USD)	Currency	18,000
Cost Date	Date	2026-11-30
Invoice Reference	Text	INV-88213

On the TV Request table, create a **rollup column** "Actual Total Cost" = SUM of Cost Entry Amount. Utilization % = Actual Total Cost ÷ Approved Amount × 100, calculated in Power BI or in a formula column.

Example: $37,800 \div 39,500 \times 100 = 95.7\%$, shown as 96%. Under 100% means the request stayed within its budget.

6. Request Comment (tvr_requestcomment) – one comment or question

Column	Type	Sample
Comment	Multiline	Please attach the wafer map.
TV Request	Lookup	TV-2026-00015
Stage	Choice (same values as Phase)	APPG Review
Comment Type	Choice: General, More Info Request, Requester Response, Decision	More Info Request
Created By / Created On	Filled by the system	David Chen / 2026-10-13

7. Request Document (tvr_requestdocument) - the file's details only

The file itself lives in SharePoint. This row only remembers its name, type, folder and link, so the app can list files quickly and reports can count them.

Column	Type	Sample
File Name	Text	Etch_Test_Spec_v1.pdf
TV Request	Lookup	TV-2026-00015
Document Type	Choice: Technical Specification, Supporting Document, Quote, Approval Evidence, Progress Report, Final Report	Technical Specification
SharePoint URL	URL	Link to the file
Folder	Text	02-Technical Documents
Uploaded On	Date and time	2026-10-12

8. Status History (tvr_statushistory) - the readable audit trail

Column	Type	Sample
Name	Text	TV-2026-00015: Submitted → Under APPG Review
TV Request	Lookup	TV-2026-00015
From Status / To Status	Choice (TVR Status)	Submitted / Under APPG Review
Changed By	Lookup → User	David Chen
Changed On	Date and time	2026-10-13 09:12
Remarks	Multiline	Picked up for review

Why have this when Dataverse auditing exists? Audit history is technical and hard to report on. This table gives one clean row per move, which the History tab and the Power BI bottleneck chart use.

9. Notification Log (tvr_notificationlog) – every email the system tried to send

Column	Type	Sample
Subject	Text	[TVRMS] TV-2026-00015 submitted
TV Request	Lookup	TV-2026-00015
Template Code	Text	REQ_SUBMITTED
Sent To	Text	priya.sharma@lam.com
Sent On	Date and time	2026-10-12 10:06
Result	Choice: Sent, Failed	Sent
Error Message	Multiline	(empty)

Use case: Priya says "I never got the email." The admin opens this log and sees whether it was Sent or Failed, and why.

10. Error Log (tvr_errorlog) – one failed flow run

Column	Type	Sample
Flow Name	Text	F01 - On Request Submitted
TV Request	Lookup	TV-2026-00015
Error	Multiline	SharePoint: folder already exists
Run URL	URL	Link to the failed run
Status	Choice: New, Fixed, Ignored	New

This supports "Exception Remediation" on slide 5: every failure is written down, fixed, and marked Fixed.

The two safety settings on the 10 child links

For every child → TV Request relationship, set the behaviour like this:

- **Delete = Restrict.** Dataverse refuses to delete a request that still has child rows. Example: an admin tries to delete TV-2026-00015; Dataverse stops it because the request has options, approvals and history. This protects the 7-year history.

- **Assign and Share = Cascade All.** When the request is shared with someone, its child rows are shared too. Example: the funding flow shares TV-2026-00015 with Maria, so she can also see its 3 feasibility options.

Steps: open the TV Request table → **Relationships** → open the one-to-many link (for example to Feasibility Option) → **Advanced options** → **Type of behaviour = Custom** → set Delete = Restrict, Assign = Cascade All, Share = Cascade All, Unshare = Cascade All → Save.

One extra check that the guide does not mention: log in as Priya and open the Feasibility tab of her request. If she cannot see the options that David created, also set **Reparent = Cascade All**. This gives the owner of a request read access to its child rows.

Four validation rules on the server side

The Canvas app checks the form, but data can also come from the admin app, an import or a flow. Rules inside Dataverse protect the data from every entry point.

1. If Request Type = Repeat → Previous Request is required.
2. If Request Type = Recurring → Recurrence Frequency is required.
3. Required By Date must be after today.
4. Estimated Budget minimum value = 0 – this is a column setting (Minimum value), not a business rule.

Steps for rules 1 and 2: TV Request table → **Business rules** → **New** → add a Condition (Request Type equals Repeat) → add an Action. Set the rule's Scope to **Entity (Table)** so it also runs outside model-driven forms, and test it from the Canvas app.

For rule 3: business rules have no built-in "today" value to compare with. Keep this check in the Canvas app, where the Submit button already has it (section 12).

How to load the sample data

The files must be loaded in this order, because a lookup can only find a parent that already exists:

1. departments.csv
2. locations.csv
3. research_institutions.csv
4. cost_centers.csv (needs departments and users)
5. app_settings.csv
6. email_templates.csv
7. tv_requests.csv (needs departments, locations, cost centers and users)

8. feasibility_options.csv (needs requests and institutions)

9. milestones.csv (needs requests)

10. cost_entries.csv (needs requests)

Example of a wrong order: you load tv_requests.csv before cost_centers.csv. The row for TV-2026-00015 says "CC-4100 Etch R&D", the import cannot find that cost center, and the row fails.

Steps: open the table → **Import** → **Import data from Excel/CSV** → map each file column to a table column → Import. The people in the files (for example maria.lopez@lam.com) must already exist as users in the environment; they come from Entra ID.

9. Doc 04 – SharePoint document management

Every request gets its own folder in one SharePoint library, created automatically on submit, with 6 fixed sub-folders. The library keeps old versions of files, has its own permissions, and keeps files for 7 years. Users never upload into SharePoint directly; they upload in the app, and a flow puts the file in the right folder.

Slide 3 asks for exactly this: "an automatically created TVID workspace, predefined folder structure, versioning, permissions, and seven-year retention".

1. The folder structure

```
Site:      https://lam.sharepoint.com/sites/TVRMS
Library:   TV Requests
├─ TV-2026-00015          ← created on submit (the "TVID workspace")
│   ├── 01-Request
│   ├── 02-Technical Documents
│   ├── 03-Feasibility & Quotes
│   ├── 04-Funding Approval
│   ├── 05-Execution
│   └─ 06-Final Report & Closure
```

Sub-folder	What goes in it	Example
01-Request	The request summary PDF (while the request is still open)	TV-2026-00015_Summary_20261020.pdf
02-Technical Documents	The mandatory specification and drawings	Etch_Test_Spec_v1.pdf
03-Feasibility & Quotes	Quotes from labs and vendors	Stanford_Quote_Q-1182.pdf

Sub-folder	What goes in it	Example
04-Funding Approval	Approval evidence	TV-2026-00015_Funding_Approval.pdf
05-Execution	Progress reports and test data	Progress_Report_Dec.pptx
06-Final Report & Closure	Final report and closure summary PDF	Final_Report.pdf

The folder names are an ASSUMPTION (gap G9). Confirm them with the business.

2. Library settings (do once in each environment)

a) Versioning. Library → Settings → Versioning settings:

- **Create major versions = Yes**, and keep about 50 versions.
- **Require check out = No**.

What versioning does: when someone uploads a file with the same name again, SharePoint keeps the old file as an older version. Example: Etch_Test_Spec_v1.pdf is uploaded on 12-Oct and again on 15-Oct. The library shows the newest file, and "Version history" still has the first one, which you can open or restore.

Why check-out must stay off: with check-out on, a file uploaded by a flow can stay checked out to the service account, and other people cannot see it.

b) Metadata columns. Metadata = labels attached to each file, used for search, filters and retention. Add 3 columns to the library:

Column	Type	Example value
TVID	Single line of text	TV-2026-00015
DocumentType	Choice: Technical Specification, Supporting Document, Quote, Approval Evidence, Progress Report, Final Report	Technical Specification
Phase	Choice (the 6 phases)	Initiation

c) Permissions. By default a library uses the site's permissions ("inherits" them). Here you stop that and give the library its own list:

1. Library settings → **Permissions for this document library** → **Stop Inheriting Permissions**.
2. SG-TVRMS-Admins → **Full Control**.

3. The service account svc-tvrms → **Full Control** (the flows use it).
4. SG-TVRMS-APPG → **Contribute** (add and edit files).
5. SG-TVRMS-Users → **Read** (ASSUMPTION: a simple model).

In this simple model every user can read every request folder. Requesters can still upload through the app, because flow F09 saves the file with the service account.

The stricter option: flow F01 breaks inheritance on each new TVID folder and gives Edit only to the requester and APPG (F01 step 6). Then Priya sees only her own folders. It is safer but more work, so decide it in the workshop.

d) One rule for users: upload through the app, never directly in SharePoint. The app path fills the metadata and creates the Request Document row; a direct upload does neither.

3. Seven-year retention

Retention means Microsoft 365 keeps a file for a fixed time, even if someone deletes it. This is set by the Microsoft 365 compliance admin in **Microsoft Purview**, not by the developer.

1. Purview portal → **Data lifecycle management** → **Retention labels** → **Create label**.
2. Name: **TVRMS-7-Years**. Retain items for **7 years** from when they were created (or labelled).
3. At the end choose **Start a disposition review**. A person then decides what to delete, which is safer than automatic deletion.
4. Publish the label, then auto-apply it or set it as the default label of the TV Requests library (Library settings → **Apply label to items in this list or library**).

For the Dataverse records the guide uses two other protections: requests are never deleted (Delete = Restrict, section 8), and audit logs are kept for 7 years (section 5).

4. How a file travels from the app to SharePoint

A Canvas app cannot easily write a file into a SharePoint folder by itself. So the app hands the file to a flow:

```
Canvas App (attachment control)
└─> Flow "F09 - Upload Document" (Power Apps V2 trigger; inputs: TVID,
DocType, Folder, File)
    └─ Create file in /TV Requests/{TVID}/{Folder}
    └─ Update file properties (TVID, DocumentType, Phase)
    └─ Add a "Request Document" row in Dataverse with the file link
```

Example: Priya attaches Etch_Test_Spec_v1.pdf and clicks Submit.

1. The app sends TVID = TV-2026-00015, DocType = Technical Specification, Folder = 02-Technical Documents, and the file to F09.
2. F09 creates /TV Requests/TV-2026-00015/02-Technical Documents/Etch_Test_Spec_v1.pdf.
3. F09 sets the file labels: TVID = TV-2026-00015, DocumentType = Technical Specification, Phase = Initiation.
4. F09 adds a Request Document row, so the file appears on the Documents tab of the request.

The full flow is in section 15.

5. Which document goes where (sample story)

Document	Uploaded by	Phase	Folder
Etch_Test_Spec_v1.pdf	Requester	Initiation	02-Technical Documents
Wafer_Map.xlsx	Requester	Initiation (after "More info")	02-Technical Documents
Stanford_Quote_Q-1182.pdf	APPG	Internal Alignment 1	03-Feasibility & Quotes
TV-2026-00015_Funding_Approval.pdf	Flow	Funding Approval	04-Funding Approval
Progress_Report_Dec.pptx	APPG	Execution	05-Execution
Final_Report.pdf	APPG	Closure	06-Final Report & Closure
TV-2026-00015_Summary.pdf	Flow	Closure	06-Final Report & Closure

10. Doc 05 – Security (role-based access)

Security works at 3 levels, and only the middle one really protects the data:

1. **Entra ID** – who can sign in at all (the security group on the environment).
2. **Dataverse security roles** – which rows a person can read or change. This is the real protection.

3. **Canvas app** – which buttons and screens a person sees. This is only for a clean user experience.

Think of an office building. Entra ID is the guard at the gate who checks your badge. Dataverse roles are the locks on each room. The Canvas app is the signs on the doors: removing a sign does not lock the door.

Slide 3 asks for role-based access for Requesters, APPG users, Cost Center Owners, agreed stakeholders and system admins. Slide 9 asks to prevent unauthorized access to records, documents and functions.

1. The 6 privileges and the 4 access levels

A security role says, for each table, what a person may do (the privilege) and on which rows (the level).

Privilege	Simple meaning	Example
Create	Make new rows	Priya creates a request
Read	See rows	David sees all requests
Write	Change rows	Priya edits her draft
Delete	Remove rows	Nobody deletes requests (7-year history)
Append	Fill a lookup on my row, pointing to another row	Priya sets Department on her request
Append To	Let other rows point to this row	Department must allow requests to point to it

Append and Append To always work as a pair. To pick "Process Engineering" on her request, Priya needs **Append** on TV Request and **Append To** on Department.

Level	Which rows	Shown in the PDF as
Organization (Org)	All rows in the environment	green circle
Business Unit	Rows of the person's business unit	yellow circle (not used here)
User	Rows the person owns, plus rows shared with the person or their team	blue circle

Level	Which rows	Shown in the PDF as
None	No rows	red cross

2. The 5 roles

Where: Power Platform admin center → environment → **Settings** → **Users + permissions** → **Security roles**. Tip: copy the built-in "Basic User" role and change the copy. Add every role to the TVRMS solution so it moves to TEST and PROD.

TVR Requester (Priya)

Table	Create	Read	Write	Delete	Append	Append To
TV Request	User	User	User	None	User	User
Request Comment, Request Document	User	User	User	None	User	User
Feasibility, Funding, Milestone, Risk, Cost	None	User	None	None	None	None
Status History	None	User	None	None	None	None
Department, Location, Cost Center, Research Institution, Email Template, App Setting	None	Org	None	None	None	Org

Result: Priya sees only her own requests, plus any request shared with her.

TVR APPG Reviewer (David)

Table	Create	Read	Write	Delete	Append	Append To
TV Request and all child tables	Org	Org	Org	None	Org	Org
Master tables	None	Org	None	None	None	Org
Notification Log, Error Log	None	Org	None	None	None	None

TVR Cost Center Owner (Maria)

Table	Read	Write	Note
TV Request	User	None	Sees only requests the funding flow shares with her
Funding Approval	User	User	Can record a decision
Child tables	User	None	Shared with her through "Cascade Share"
Master tables	Org	None	

TVR Stakeholder (Tom) – Read at User level on TV Request and its children, only rows shared with him. No write.

TVR Admin – Organization level on everything, can write master data, but no Delete on TV Request (because of retention). Also give the built-in System Administrator role to 1 or 2 IT people.

Watch out – two traps in this role design

- **Status History.** The Submit and Resubmit buttons (section 12) write a Status History row as the signed-in user. The Requester role has Create = None on that table, so Priya's save will fail. Fix: give the Requester role Create and Append at User level on Status History, or let a flow write the history rows.
- **Column security on the status.** The guide suggests (optionally) locking tvr_status so only APPG and Admin can write it. Column security locks the whole column, not single values. Then Priya's own Submit, Resubmit and Cancel also fail. If you lock it, run those 3 actions through a flow that uses the service account. Locking tvr_approvedamount is safe, because requesters never write it.

3. Row-level security with an Access Team

Idea. An access team is a tiny team attached to one single request. Adding a person to it gives that person access to that one request only. This is how Maria (CCO) and Tom (stakeholder) see TV-2026-00015 without seeing any other request.

Steps (no code):

1. Turn on access teams for the TV Request table (Table properties → Advanced options → "Have an access team").
2. Create an access team template: Name = **TVR Request Team**, Table = TV Request, rights = Read, Append, Append To. In the Power Platform admin center this sits under the environment's Settings, in the templates area ("Access team templates").
3. When the funding flow starts, it adds the Cost Center Owner to the request's access team (action AddUserToRecordTeam – see section 22 for where to find it).
4. APPG adds agreed stakeholders from the Request Detail screen with a button that runs flow F10.

Sample: the access team of TV-2026-00015 = Maria Lopez (CCO) and Tom Baker (stakeholder).

This is why the CCO and Stakeholder roles have Read at **User** level: the role allows reading, and the access team decides which single request.

4. Give roles to groups, not to people

Create a Dataverse team for each Entra group: admin center → **Teams** → **New team** → **Team type = Microsoft Entra ID Security Group**, then assign the role to the team.

Dataverse team	Entra group	Role
TVRMS Requesters	SG-TVRMS-Requesters	TVR Requester
TVRMS APPG	SG-TVRMS-APPG	TVR APPG Reviewer
TVRMS Cost Center Owners	SG-TVRMS-CCOwners	TVR Cost Center Owner
TVRMS Admins	SG-TVRMS-Admins	TVR Admin

Now IT only adds a person to the Entra group, and the access follows automatically. Example: a new engineer, Ravi, joins. IT adds him to SG-TVRMS-Users and SG-TVRMS-Requesters. Ravi can now open the app and create requests.

5. Showing and hiding things in the Canvas app

The app works out the user's role once, in **App.Formulas** (named formulas that recalculate by themselves):

```
CurrentUserRecord = LookUp(Users, 'Primary Email' = User().Email);
IsAPPG = !IsEmpty(Filter(LookUp(Teams, 'Team Name' = "TVRMS APPG").'Users
(teammembership_association)', 'Primary Email' = User().Email));
IsAdmin = !IsEmpty(Filter(LookUp(Teams, 'Team Name' = "TVRMS Admins").'Users
(teammembership_association)', 'Primary Email' = User().Email));
```

Reading IsAPPG from the inside out:

1. `LookUp(Teams, 'Team Name' = "TVRMS APPG")` finds the APPG team row.
2. `.'Users (teammembership_association)'` gives the list of that team's members.
3. `Filter(..., 'Primary Email' = User().Email)` keeps only the signed-in person.
4. `IsEmpty(...)` is true when that list is empty (I am not a member); `!` flips it. So `IsAPPG = true` for David and `false` for Priya.

`CurrentUserRecord` finds the signed-in person's row in the Users table. The app uses it to fill Requester on a new request.

A simpler alternative (ASSUMPTION – fine for this size): a small "App Role Member" table with two columns, User and Role. Then:

```
IsAPPG = !IsBlank(LookUp('App Role Members', User.'Primary Email' =
User().Email && Role = 'Role (App Role Members)'.APPG))
```

Use it on any control, for example `btnApprove.Visible = IsAPPG`. Remember: hiding a button is not security. The Dataverse role is what really stops Priya.

6. Security tests (sample)

Test	Log in as	Expected result
Requester sees only own requests	Priya	Sees TV-2026-00015, not TV-2026-00016 (John's)
Requester cannot set "Funding Approved"	Priya	Button hidden, and Dataverse blocks the change
CCO sees only shared requests	Maria	Sees TV-2026-00015 after the funding flow starts
Stakeholder cannot edit	Tom	Form is read-only; a save fails
Admin edits master data	Admin	Can add a new Cost Center

The sample data has a small mismatch: in tv_requests.csv, TV-2026-00016 belongs to Priya, not John. For test 1, use John's TV-2026-00011 or TV-2026-00018 instead.

11. Doc 06 (part 1) – Canvas app: setup, start-up formulas, Home screen

The Canvas app is the main app for end users. It has 8 screens that cover all 6 phases, plus field checks, error handling and "Export to PDF" (slide 5). This part covers creating the app, the formulas that run at start-up, and the Home screen.

1. Create the app

1. `make.powerapps.com` → LAM-TVRMS-DEV → **Solutions** → **TVRMS** → **New** → **App** → **Canvas app**.
2. Name = **TV Request Management**, Format = **Tablet** (works well in Edge on a laptop).
3. Data pane → **Add data** → add these Dataverse tables: TV Requests, Feasibility Options, Funding Approvals, Milestones, Risk Issues, Cost Entries, Request Comments, Request Documents, Status Histories, Departments, Cost Centers, Locations, Research Institutions, App Settings, Users. (Add Teams too, because the IsAPPG formula reads it.)
4. **Settings** → turn on "Formula-level error management" and "Named formulas". In newer Power Apps versions these may already be on.

Naming convention – start every control name with a short type code, so formulas are easy to read:

Code	Control type	Example
scr	Screen	scrHome
gal	Gallery (list)	galMyRequests
frm	Form	frmRequest
btn	Button	btnSubmit
txt	Text input	txtSearch
dd	Dropdown	ddStatusFilter
cmb	Combo box	cmbPrevious
dp	Date picker	dpRequiredBy
lbl	Label	lblMyOpen
att	Attachment control	attTechDocs

2. Global choices

A **global choice** is one list of values that many columns can share. Create these 4 first (Solution → **New** → **Choice**): **TVR Status**, **TVR Phase**, **TVR Request Type**, **TVR Priority**. Then, when you create a choice column, pick "Sync with global choice" and choose the list.

Why it matters in formulas: with a global choice you write `'TVR Status'.Submitted`. With a local choice you would write the longer `'Request Status (TV Requests)'.Submitted`. It also lets Status History use exactly the same values as the request.

3. Start-up formulas

```
// App.Formulas
gblAppColorPrimary = ColorValue("#3B2BB3");
CurrentUserRecord = LookUp(Users, 'Primary Email' = User().Email);
IsAPPG = !IsEmpty(Filter(LookUp(Teams, 'Team Name' = "TVRMS APPG").'Users
(teammembership_association)', 'Primary Email' = User().Email));
IsAdmin = !IsEmpty(Filter(LookUp(Teams, 'Team Name' = "TVRMS Admins").'Users
(teammembership_association)', 'Primary Email' = User().Email));
```

- **gblAppColorPrimary** – the main purple colour, defined once and used by every header and button.

- **CurrentUserRecord** – the signed-in person's row in the Users table (explained in section 10).
- **IsAPPG / IsAdmin** – true or false, used to show or hide buttons.

Opening a request straight from an email (deep link). Every email contains a link like `{AppUrl}?TVID=TV-2026-00015`. The part after ? is a parameter. `Param("TVID")` reads it, so the app can open that request directly.

The guide puts this in `App.OnStart`:

```
If(!IsBlank(Param("TVID")),
    Set(varSelectedRequest, LookUp('TV Requests', TVID = Param("TVID")));
    Navigate(scrRequestDetail))
```

Correction: current Power Apps does not allow `Navigate` inside `App.OnStart`. Use the `StartScreen` property to pick the first screen, and load the request when the detail screen opens:

```
// App.StartScreen
If(!IsBlank(Param("TVID")), scrRequestDetail, scrHome)

// scrRequestDetail.OnVisible
If(IsBlank(varSelectedRequest) && !IsBlank(Param("TVID")),
    Set(varSelectedRequest, LookUp('TV Requests', TVID = Param("TVID"))))
```

Example: Maria clicks the link in her approval email. The app opens directly on TV-2026-00015 instead of the Home screen.

4. The 8 screens

#	Screen	Who uses it	Purpose
1	scrHome	Everyone	My requests, counters, "New Request" button
2	scrNewRequest	Requester	3-step wizard to create and submit
3	scrRequestDetail	Everyone (content depends on role)	Everything about one TVID, in tabs
4	scrAPPGQueue	APPG	Work queue of requests to review
5	scrFeasibility	APPG	Compare research institutions and options

#	Screen	Who uses it	Purpose
6	scrFunding	APPG, CCO	Send for funding, see approval history
7	scrExecution	APPG	Milestones, risks and issues, costs, progress
8	scrClosure	APPG	Final outcome, lessons learned, close request

5. Screen 1 - Home (scrHome)

Layout: a header bar (logo, title, user name), 4 counter tiles, a filter row (search box and status dropdown), and a list (gallery) of requests.

Counter tiles. The guide gives the formulas for 2 of the 4 tiles; choose the other 2 in the workshop.

```
// lblMyOpen.Text - my requests that are not finished
CountRows(Filter('TV Requests', Requester.'Primary Email' = User().Email,
    !('Request Status' in ['TVR Status'.Closed, 'TVR Status'.Cancelled, 'TVR
    Status'.Rejected, 'TVR Status'.Funding Rejected])))

// lblNeedsMyAction.Text - requests waiting for my answer
CountRows(Filter('TV Requests', Requester.'Primary Email' = User().Email,
    'Request Status' = 'TVR Status'.More Info Required'))
```

In words: count my requests whose status is not one of the 4 final statuses; and count my requests that are waiting for my reply.

Example with the sample data, for Priya: she owns TV-2026-00003 (In Execution), TV-2026-00015 (In Execution) and TV-2026-00016 (More Info Required). So My Open = 3 and Needs My Action = 1.

The request list (galMyRequests.Items):

```
SortByColumns(
    Filter('TV Requests',
        (IsAPPG || Requester.'Primary Email' = User().Email),
        IsBlank(txtSearch.Text) || StartsWith(TVID, txtSearch.Text) ||
        StartsWith('Request Title', txtSearch.Text),
        IsBlank(ddStatusFilter.Selected.Value) || 'Request Status' =
        ddStatusFilter.Selected.Value),
    "createdon", SortOrder.Descending)
```

Read it line by line:

1. APPG sees every request; anyone else sees only their own.
2. If the search box is empty, keep everything; otherwise keep rows whose TVID or title starts with the typed text.
3. If no status is picked, keep everything; otherwise keep only that status.
4. Show the newest request first.

Example: Priya types "TV-2026-0001". She sees TV-2026-00015 and TV-2026-00016. She does not see John's TV-2026-00011, because it is not hers.

Dataverse security already limits the rows; this filter only makes the list cleaner.

Inside each list row, show TVID, Title, Status (as a coloured "pill"), Required By and Phase. The pill colour:

```
// lblStatusPill.Fill
Switch(ThisItem.'Request Status',
    'TVR Status'.More Info Required', ColorValue("#F4B400"),
    'TVR Status'.Rejected,           ColorValue("#D93025"),
    'TVR Status'.Funding Rejected',  ColorValue("#D93025"),
    'TVR Status'.Closed,             ColorValue("#188038"),
    ColorValue("#1A73E8"))
```

Yellow = waiting for the requester, red = rejected, green = closed, blue = everything else (the last value is the default).

Two buttons on Home:

- `galMyRequests.OnSelect = Set(varSelectedRequest, ThisItem);`
`Navigate(scrRequestDetail)` - clicking a row remembers that request and opens its detail screen.
- `btnNewRequest.OnSelect = NewForm(frmRequest); Set(varStep, 1);`
`Navigate(scrNewRequest)` - puts the form in "new record" mode, starts the wizard at step 1, and opens it.

Tip: if Power Apps shows a yellow warning triangle (a "delegation" warning) on a filter, that filter may only check the first rows of the table. With about 500 requests a year it is fine at the start, but test it with more rows before go-live.

12. Doc 06 (part 2) - New Request wizard, Submit, status changes, and the other screens

This part explains the formulas that do the real work: the 3-step New Request wizard, the Submit button, one shared "change status" button that all screens use, and the APPG, Feasibility, Funding, Execution and Closure screens.

1. Screen 2 – New Request wizard (scrNewRequest)

There is one Edit Form, **frmRequest** (data source 'TV Requests'). A variable **varStep** (1, 2 or 3) decides which fields are visible.

Step	Fields (* = required)	Sample input
1 – Basic	Title*, Request Type*, Previous Request (if Repeat), Frequency (if Recurring), Department*, Location*, Cost Center*, Priority*, Required By*, Estimated Budget*	"Plasma etch uniformity test", New, -, -, Process Engineering, Fremont CA, CC-4100, High, 15-Dec-2026, 42000
2 – Details	Business Justification*, Technical Description*, Expected Outcome*	Free text
3 – Documents & Submit	Attachments (technical document*), other documents, a review summary, Save as Draft / Submit	Etch_Test_Spec_v1.pdf

How to make the steps (simple Next and Back buttons, not written out in the guide):

- Each data card's Visible = its step, for example `varStep = 1` for Title and `varStep = 2` for Business Justification.
- `btnNext.OnSelect = Set(varStep, varStep + 1)` and `btnBack.OnSelect = Set(varStep, varStep - 1)`.

Show a field only when it is needed:

```
// Data card "Previous Request".Visible
ddRequestType.Selected.Value = 'TVR Request Type'.Repeat
// Data card "Recurrence Frequency".Visible
ddRequestType.Selected.Value = 'TVR Request Type'.Recurring
```

(ddRequestType is the Request Type control inside the form, renamed. Add `varStep = 1` && in front so these cards also hide on steps 2 and 3.)

Repeat request – copy details from the old TVID:

```
// cmbPrevious.OnChange
If(!IsBlank(cmbPrevious.Selected),
    Set(varCopyFrom, cmbPrevious.Selected);
    Notify("Details copied from " & varCopyFrom.TVID & ". Please review.",
    NotificationType.Information))

// Default of each text input, for example Business Justification
If(!IsBlank(varCopyFrom), varCopyFrom.'Business Justification',
    Parent.Default)
```

Example: Priya picks Repeat and chooses TV-2026-00003. The justification and description of the old request appear in the boxes, and she only changes the liner version.

Tip: clear the helper variables when a new request starts. Change btnNewRequest to `NewForm(frmRequest); Set(varStep, 1); Set(varDraft, Blank()); Set(varCopyFrom, Blank()); Navigate(scrNewRequest)`. Otherwise an old draft or old copied text can leak into the next new request.

The attachment control trick. Power Apps has no stand-alone attachment control in the Insert menu; it only exists inside a form's Attachments card. So add a SharePoint or Dataverse form once, copy its Attachments control, paste it on scrNewRequest, rename it **attTechDocs**, and delete the helper form. Set `MaxAttachments = 10` and `MaxAttachmentSize = 25 (MB)`.

2. The Submit button, block by block (btnSubmit.OnSelect)

Block 1 – Checks. An If with several conditions works like "if ... else if ... else". The first true condition shows its error and stops.

```
If(
    !frmRequest.Valid, Notify("Please fill all fields marked with *",
    NotificationType.Error),
    CountRows(attTechDocs.Attachments) = 0, Notify("Please attach at least one
    technical specification document.", NotificationType.Error),
    dpRequiredBy.SelectedDate <= Today(), Notify("'Required By' date must be a
    future date.", NotificationType.Error),
    ddRequestType.Selected.Value = 'TVR Request Type'.Repeat &&
    IsBlank(cmbPrevious.Selected), Notify("Please select the previous request for
    a Repeat request.", NotificationType.Error),
    // all checks passed: blocks 2 to 4 go here
)
```

- `frmRequest.Valid` is true only when every required field is filled.
- Example: Priya leaves Title empty → "Please fill all fields marked with *" and nothing is saved (test TC-01).

Block 2 – Save as Draft first, to get a TVID. The TVID is created only when the row is saved, and the files need the TVID to find their folder.

```
Set(varNew, Patch('TV Requests',
  If(IsBlank(varDraft), Defaults('TV Requests'), varDraft),
  frmRequest.Updates,
  { Requester: CurrentUserRecord, 'Request Status': 'TVR Status'.Draft,
    'Current Phase': 'TVR Phase'.Initiation }));
```

Patch(table, which row, changes...): Defaults(...) = a new empty row, or varDraft = the draft saved earlier; frmRequest.Updates = everything typed in the form; the last record adds the requester, status and phase. The saved row, with its new TVID, is kept in varNew.

The guide then writes IfError(varNew, Notify("Could not save: " & FirstError.Message, NotificationType.Error); Exit()). **Correction:** in Power Apps, Exit() closes the whole app; it does not just stop the formula. Use this instead:

```
Set(varNew, IfError(Patch( ...same as above... ),
  Notify("Could not save: " & FirstError.Message, NotificationType.Error);
  Blank()));
If(!IsBlank(varNew),
  // blocks 3 and 4 go here
)
```

Block 3 – Upload each file through flow F09.

```
ForAll(attTechDocs.Attachments As f,
  'F09-UploadDocument'.Run(varNew.TVID, "Technical Specification", "02-
  Technical Documents",
    { file: { name: f.Name, contentBytes: f.Value } }));
```

ForAll repeats the call once per attached file. Each call sends the TVID, the document type, the folder and the file. When you type .Run(, Power Apps shows the exact inputs the flow expects; follow that hint for the file part.

Block 4 – Set the status to Submitted (this starts flow F01), write history, finish.

```
Patch('TV Requests', varNew, { 'Request Status': 'TVR Status'.Submitted,
  'Current Phase': 'TVR Phase'.APPG Review',
  'Submitted On': Now(), 'Status Changed On': Now() });
Patch('Status Histories', Defaults('Status Histories'),
  { Name: varNew.TVID & ": Draft → Submitted", 'TV Request': varNew,
  'From Status': 'TVR Status'.Draft, 'To Status': 'TVR Status'.Submitted,
  'Changed On': Now(), Remarks: "Submitted by requester" });
Notify("Request " & varNew.TVID & " submitted successfully.",
  NotificationType.Success);
Reset(attTechDocs); Set(varDraft, Blank());
Navigate(scrHome)
```

Two small improvements: add `'Changed By': CurrentUserRecord` to the history row (the guide leaves it empty), and remember the Status History permission trap from section 10.

Save as Draft (btnSaveDraft.OnSelect) – the same Patch with status Draft, and only one check (Title). The result goes into varDraft, so the next save updates the same row instead of making a new one.

```
If(IsBlank(frmRequest.Updates.'Request Title'),
  Notify("Please enter a title.", NotificationType.Error),
  Set(varDraft, Patch('TV Requests', If(IsBlank(varDraft), Defaults('TV
Requests'), varDraft),
  frmRequest.Updates, { Requester: CurrentUserRecord, 'Request Status':
'TVR Status'.Draft, 'Current Phase': 'TVR Phase'.Initiation }));
  Notify("Draft saved as " & varDraft.TVID, NotificationType.Success))
```

A draft gets a TVID but no folder and no email, because F01 starts only at Submitted (test TC-07). Keep the required columns as "Business required" in Dataverse (not "System required"), so a half-filled draft can still be saved.

3. One shared "change status" button (btnDoStatusChange)

Many buttons change the status. Instead of writing the same logic 12 times, one hidden button on the detail screen does it. Other buttons fill two variables and "press" it:

```
Set(varNewStatus, 'TVR Status'.More Info Required');
Set(varRemarks, txtReviewComment.Text);
Select(btnDoStatusChange)
```

What the hidden button does:

```

With({ oldStatus: varSelectedRequest.'Request Status' },
  Set(varSelectedRequest, Patch('TV Requests', varSelectedRequest,
    { 'Request Status': varNewStatus, 'Status Changed On': Now(),
      'Current Phase': Switch(varNewStatus,
        'TVR Status'.Internal Alignment 1, 'TVR Phase'.Internal
Alignment',
        'TVR Status'.Internal Alignment 2, 'TVR Phase'.Internal
Alignment',
        'TVR Status'.Pending Funding Approval, 'TVR Phase'.Funding
Approval',
        'TVR Status'.Funding Approved, 'TVR Phase'.Funding Approval',
        'TVR Status'.In Execution, 'TVR Phase'.Execution,
        'TVR Status'.On Hold, 'TVR Phase'.Execution,
        'TVR Status'.Execution Completed, 'TVR Phase'.Closure,
        'TVR Status'.Closed, 'TVR Phase'.Closure,
        varSelectedRequest.'Current Phase') }));
Patch('Status Histories', Defaults('Status Histories'),
  { Name: varSelectedRequest.TVID & ": " & Text(oldStatus) & " → " &
Text(varNewStatus),
    'TV Request': varSelectedRequest, 'From Status': oldStatus, 'To
Status': varNewStatus,
    'Changed On': Now(), Remarks: varRemarks });
If(!IsBlank(varRemarks),
  Patch('Request Comments', Defaults('Request Comments'),
    { Comment: varRemarks, 'TV Request': varSelectedRequest, Stage:
varSelectedRequest.'Current Phase' })))

```

1. `With({oldStatus: ...})` remembers the old status before it changes.
2. The first Patch saves the new status and time, and works out the phase. Statuses not listed (Under APPG Review, More Info Required, Rejected, Cancelled) keep the current phase – the last line of the Switch.
3. The second Patch writes the Status History row, for example "TV-2026-00015: Under APPG Review → More Info Required".
4. If there is a remark, it is also saved as a Request Comment.
5. Flow F02 notices the new status and sends the right email (section 14).

4. Screen 3 – Request Detail (scrRequestDetail)

Top of the screen: TVID, Title, the status pill, and a progress bar of 6 chevrons with the current phase highlighted. Below it, tabs (a horizontal gallery of tab names plus a variable `varTab`):

Tab	Content	Who can edit
Summary	The request form in display mode	Requester, only in Draft or More Info Required
Comments	List of comments plus an “add” box	Everyone with access
Documents	List of files (click opens SharePoint) plus upload	Requester, APPG
Feasibility	The options (read-only for the requester)	APPG
Funding	Approval history	APPG sends; CCO reads
Execution	Milestones, risks and issues, costs	APPG
Closure	Outcome, lessons learned, reuse	APPG
History	Status History, newest first	Read only

Action buttons – each is visible only for the right role and status:

Button	Visible when	Sets status to
Start Review	IsAPPG and status = Submitted	Under APPG Review (and APPG Owner = me)
Request More Info	IsAPPG and status = Under APPG Review	More Info Required (comment mandatory)
Reject	IsAPPG and status = Under APPG Review or Internal Alignment 1	Rejected (reason mandatory)
Resubmit	I am the requester and status = More Info Required	Under APPG Review
Move to Internal Alignment	IsAPPG and status = Under APPG Review	Internal Alignment 1
Confirm Option (Alignment 2)	IsAPPG and status = Internal Alignment 1 and an option is selected	Internal Alignment 2
Send for Funding Approval	IsAPPG and status = Internal Alignment 2	Pending Funding Approval (flow F04 starts)
Start Execution	IsAPPG and status = Funding Approved	In Execution

Button	Visible when	Sets status to
Put On Hold / Resume	IsAPPG (status In Execution or On Hold)	On Hold / In Execution
Complete Execution	IsAPPG and status = In Execution	Execution Completed
Close Request	IsAPPG and status = Execution Completed and a final report is uploaded	Closed (flow F07 runs)
Cancel	(I am the requester and status = Draft, Submitted or More Info Required) or IsAPPG	Cancelled
Export to PDF	Everyone	Runs flow F06

Reject with a mandatory reason:

```
// btnReject.OnSelect
If(IsBlank(txtReviewComment.Text),
    Notify("Please enter the rejection reason.", NotificationType.Error),
    Patch('TV Requests', varSelectedRequest, { 'Rejection / Cancellation
Reason': txtReviewComment.Text });
    Set(varNewStatus, 'TVR Status'.Rejected);
    Set(varRemarks, "Rejected: " & txtReviewComment.Text);
    Select(btnDoStatusChange))
```

Empty reason → error, nothing changes. With a reason → the reason is saved, then the shared button does the rest. Example: David rejects TV-2026-00011 with "Duplicate of TV-2026-00015 scope".

Late date in red: `lblRequiredBy.Color = If(varSelectedRequest.'Required By Date' < Today() && varSelectedRequest.'Request Status' <> 'TVR Status'.Closed, Color.Red, Color.Black)` – red when the date has passed and the request is not closed.

5. Screen 4 – APPG Queue (scrAPPGQueue)

```
// galQueue.Items - requests waiting for APPG, oldest first
SortByColumns(
    Filter('TV Requests',
        'Request Status' in ['TVR Status'.Submitted, 'TVR Status'.Under APPG
Review', 'TVR Status'.Internal Alignment 1',
            'TVR Status'.Internal Alignment 2', 'TVR
Status'.Funding Approved', 'TVR Status'.In Execution',
            'TVR Status'.Execution Completed'],
        chkMyItems.Value = false || 'APPG Owner'.Primary Email = User().Email),
    "tvr_statuschangedon", SortOrder.Ascending)
```

It shows every status where APPG must act next. When the "My items" checkbox is ticked, only requests where I am the APPG Owner show. The request that has waited longest is on top.

Days waiting: `RoundDown(DateDiff(ThisItem.'Status Changed On', Now(), TimeUnit.Hours) / 24, 0)`. Example: status changed 10-Oct 09:00, now 13-Oct 15:00 → 78 hours ÷ 24 = 3.25 → 3 days.

Assign an owner: a combo box `cmbAPPGOwner` (its `Items` = the users of the APPG team), then `Patch('TV Requests', varSelectedRequest, { 'APPG Owner': cmbAPPGOwner.Selected })`.

6. Screen 5 – Feasibility (scrFeasibility)

```
// galOptions.Items - the options of this request
Filter('Feasibility Options', 'TV Request'.TV Request' =
varSelectedRequest.TV Request')
```

Here `'TV Request'.TV Request'` is the hidden unique ID (GUID) of the request, so the filter compares IDs, not names.

```
// "Select this option" button inside the gallery
ForAll(Filter('Feasibility Options', 'TV Request'.TV Request' =
varSelectedRequest.TV Request') As o,
    Patch('Feasibility Options', o, { 'Is Selected': false }));
Patch('Feasibility Options', ThisItem, { 'Is Selected': true, Recommendation:
'Recommendation (Feasibility Options).Recommended });
Set(varSelectedRequest, Patch('TV Requests', varSelectedRequest, { 'Selected
Option': ThisItem, 'Estimated Budget': ThisItem.'Quoted Cost' })))
```

1. Untick all options of this request.
2. Tick the clicked option and mark it Recommended.

3. Save it on the request as Selected Option, and copy its quoted cost into Estimated Budget.

Step 3 overwrites Priya's first estimate (42,000 becomes 39,500). If LAM wants to keep both numbers, use a separate column for the selected cost.

Sample comparison shown to APPG:

Option	Institution	Capability	Cost (USD)	Weeks	Constraint	Recommendation
A	Stanford Nanofab Lab	5	39,500	10	Lab free from Nov	Recommended
B	Fraunhofer IPMS	4	35,000	16	Shipping to Germany	Alternative
C	Internal Lab Fremont	3	22,000	20	Tool booked till Jan	Not recommended

A small column chart (cost against weeks) helps the decision. Quotes are uploaded with F09 into 03-Feasibility & Quotes.

7. Screen 6 – Funding (scrFunding)

Shows the cost center, its owner (from master data), the selected option's cost, and the approval history.

```
// lblCCOwner.Text - follows two lookups: request → cost center → owner
varSelectedRequest.'Cost Center'. 'Cost Center Owner'. 'Full Name'
// galFundingHistory.Items - newest first
SortByColumns(Filter('Funding Approvals', 'TV Request'. 'TV Request' =
varSelectedRequest.'TV Request'), "createdon", SortOrder.Descending)
```

The **Send for Funding Approval** button sets the status Pending Funding Approval. Flow F04 then creates the approval for the CCO, which appears in Outlook, Teams and the Approvals app. The CCO can also open the request from the email link and see everything read-only.

8. Screen 7 – Execution (scrExecution)

Three parts:

1. **Milestones** – an editable list (planned and actual dates, % complete). When a milestone is saved, the overall progress is the average:

```
Patch('TV Requests', varSelectedRequest,
  { 'Overall Progress %': Round(Average(Filter('Milestones', 'TV Request'. 'TV Request' = varSelectedRequest.'TV Request'), '% Complete'), 0) })
```

Example from the sample data: 4 milestones at 100, 60, 0 and 0 → average 40 → Overall Progress 40%.

2. **Risks / Issues / Exceptions** – add and update. A High severity row makes flow F02b send an exception email.

3. **Costs** – add cost entries and show the actual cost against the approved amount:

```
lblUtil.Text = Text(varSelectedRequest.'Actual Total Cost' /
varSelectedRequest.'Approved Amount', "0%")
```

Example: $37,800 \div 39,500 = 0.957 \rightarrow$ shown as "96%". Show it in red above 100%. The screen also has the Expected Completion Date picker and an "Add update / comment" box.

Watch out: Actual Total Cost is a rollup column, and Dataverse recalculates rollups on a schedule, not at once. To show the new total right after a cost is added, calculate it in the app: `Sum(Filter('Cost Entries', 'TV Request'. 'TV Request' = varSelectedRequest.'TV Request'), 'Amount (USD)')`.

9. Screen 8 – Closure (scrClosure)

Fields: Final Outcome*, Lessons Learned*, Reuse Recommendation*, Final Report upload* (goes to 06-Final Report & Closure), and the final cost (read only).

```
// btnCloseRequest.OnSelect
If(
  IsBlank(txtOutcome.Text) || IsBlank(txtLessons.Text) ||
  IsBlank(ddReuse.Selected),
    Notify("Outcome, lessons learned and reuse recommendation are
mandatory.", NotificationType.Error),
    IsEmpty(Filter('Request Documents', 'TV Request'. 'TV Request' =
varSelectedRequest.'TV Request',
  'Document Type' = 'Document Type (Request Documents)'. 'Final Report')),
    Notify("Please upload the final report first.", NotificationType.Error),
    Patch('TV Requests', varSelectedRequest, { 'Final Outcome':
txtOutcome.Text, 'Lessons Learned': txtLessons.Text,
  'Reuse Recommendation': ddReuse.Selected.Value, 'Closed On': Now() });
    Set(varNewStatus, 'TVR Status'.Closed); Set(varRemarks, "Request closed");
    Select(btnDoStatusChange))
```

1. All three text fields filled? If not, error.

2. Is there a document of type "Final Report"? If not, error (test TC-22).
3. Save the closure fields and the Closed On time, then set the status Closed. Flow F07 then sends the completion emails.

10. Export to PDF

```
// btnExportPdf.OnSelect
Set(varPdf, 'F06-GeneratePDF'.Run(varSelectedRequest.TVID));
Launch(varPdf.fileurl)
```

The app sends the TVID to flow F06. The flow builds the PDF, saves it in SharePoint and returns its link; Launch opens that link.

11. Error handling rules

- Wrap every Patch: `IfError(Patch(...), Notify(FirstError.Message, NotificationType.Error))`.
- Stop double clicks: `btnSubmit.DisplayMode = If(varSaving, DisplayMode.Disabled, DisplayMode.Edit)`. Put `Set(varSaving, true)` at the start of the OnSelect and `Set(varSaving, false)` at the end.
- Show a spinner while `varSaving = true`.
- App.OnError catches anything unexpected: `Notify("Something went wrong. Please try again or contact support.", NotificationType.Error)`.

12. Share the app

Solution → the app → **Share** with the Entra group SG-TVRMS-Users, not with single people. Then copy the app's play link into the environment variable `tvr_AppUrl`, because every email link is built from it.

13. Doc 07 – Model-driven app (for admins)

The admin app lets IT manage master data, settings and email templates, and check logs and audit history. Slide 5 says: "Model Driven App (Admin users) – Master Data management, View Notification Logs".

Idea. In a model-driven app you do not draw screens. You choose tables, views (saved lists) and forms, and Power Apps builds the screens for you. It takes far less work than a Canvas app and is ideal for admin tasks.

1. Create the app

1. Solution TVRMS → **New** → **App** → **Model-driven app**.
2. Name = **TVRMS Administration**.
3. Build the left menu (the "sitemap"). An **Area** is a top-level section, a **Group** is a heading inside it, and each **Page** opens one table.

Area	Group	Pages (tables)
Requests	Requests	TV Requests (all views), Feasibility Options, Funding Approvals
Requests	Execution	Milestones, Risk Issues, Cost Entries
Master Data	Organisation	Departments, Cost Centers, Locations
Master Data	Partners	Research Institutions
Configuration	Settings	App Settings, Email Templates, Status Transitions (optional)
Monitoring	Logs	Notification Logs, Error Logs, Status Histories
Monitoring	Audit	Audit Summary (link to the Dataverse audit history)

4. Share the app only with the TVRMS Admins team (role TVR Admin).

2. Views to create

A view is a saved list: a filter plus the columns to show, like a saved filter in Excel.

Table	View name	Filter	Columns
TV Requests	Active Requests	Status not in (Closed, Cancelled, Rejected, Funding Rejected)	TVID, Title, Requester, Status, Phase, Status Changed On, Required By
TV Requests	Overdue Requests	Required By < today AND Status not Closed	TVID, Title, Requester, Required By, APPG Owner
Cost Centers	Active Cost Centers	Is Active = Yes	Code, Name, Owner, Owner's Manager
Notification Logs	Failed Notifications	Result = Failed	Subject, Sent To, Sent On, Error

Table	View name	Filter	Columns
Error Logs	Open Errors	Status = New	Flow Name, TVID, Error, Run URL

Example with the sample data: on 20-Dec-2026 the Overdue Requests view would list TV-2026-00003 (needed by 30-Nov) and TV-2026-00015 (needed by 15-Dec).

Small improvement: in Overdue Requests, also exclude Cancelled, Rejected and Funding Rejected. Otherwise a cancelled request whose date has passed (for example TV-2026-00009 after 31-Dec-2026) shows up as "overdue".

3. Forms

- **Cost Center main form:** Code, Name, Owner, Owner's Manager, Department, Is Active. Add a business rule that makes Owner required, because the funding flow cannot route an approval without an owner.
- **App Setting form:** Key, Value, Description. The Key must be read-only after the row is created, because flows find settings by their key. One way: a business rule "if Name contains data → lock Name".
- **Email Template form:** Code, Subject, Body (use the rich text control so admins can format the email), Is Active.
- **TV Request main form (admin view):** all fields in tabs, a **sub-grid** for each child table (a sub-grid is a small list of child rows inside the form), and the built-in **Audit History** tab.

4. Typical admin tasks (use cases)

Use case	Steps	Sample
Finance creates a new cost center	Master Data → Cost Centers → New → fill → Save	CC-4300 "Deposition R&D", Owner = Kevin Brown
A cost center owner changes	Open CC-4100 → change Owner → Save. New funding requests go to the new owner	Maria Lopez → Anna White
Change the reminder days	Configuration → App Settings → ReminderAfterDays → 2 → Save	No code change needed
An email failed	Monitoring → Failed Notifications → read the error → fix the address → re-run from the flow's run history	"Mailbox not found"

Use case	Steps	Sample
See who changed a request	Open the request → Related → Audit History	Status changed by David Chen on 13-Oct
Stop using a vendor	Research Institution → Is Active = No	"Old Lab Inc."

Note on the owner change: approvals that are already waiting stay with the old owner. Section 18 (the support runbook) says to cancel such an approval and send it again.

Never delete master data – set Is Active = No. The Canvas app dropdowns show only active rows, for example:

```
cmbCostCenter.Items = Filter('Cost Centers', 'Is Active' = true)
```

So the old vendor disappears from new requests, but old requests still show it correctly.

14. Doc 08 (part 1) – Flow rules, the email building block, F01 and F02

All background work is done by about 10 cloud flows named F01 to F10. Each automation activity on slide 5 maps to one of them. A flow is a recipe: a **trigger** (what starts it) followed by **actions** (the steps).

Which flow does which slide-5 activity

Activity on slide 5	Flow
SharePoint workspace creation	F01
Submission notification	F01
Missing information notification	F02
Funding approval routing	F04
Status change notifications	F02
Reminders and escalations (+ automatic cancellation)	F05
Exception remediation	F08 pattern + Error Log table
Completion notifications	F07
Generate PDF and store	F06

Activity on slide 5	Flow
Audit trail capture	Dataverse auditing + Status History rows + Notification Log
Document upload from the app	F09

There are also F02b (alert on a high-severity issue), F10 (add a stakeholder) and an optional child flow F00 (send one email). There is no F03 – the numbering simply skips it.

6 rules for every flow

1. Create it inside the solution: **Solution → New → Automation → Cloud flow**.
2. Use the connection references and environment variables from section 5, never fixed addresses.
3. Owner = the service account svc-tvrms.
4. Name format: **TVRMS - F01 - On Request Submitted**.
5. Put the steps inside Try / Catch scopes (the F08 pattern, section 15).
6. Remember that choice values are **numbers** inside flows.

Why numbers? Dataverse stores a choice as a number; the label ("Submitted") is only for people. A flow filter must use the number. The numbers come from the publisher's "choice value prefix" and from the order in which you add the values, so check yours in the choice editor and write them down once:

Status	Value (example)
Draft	100000000
Submitted	100000001
Under APPG Review	100000002
More Info Required	100000003
Rejected	100000004
Internal Alignment 1	100000005
Internal Alignment 2	100000006
Pending Funding Approval	100000007
Funding Rejected	100000008
Funding Approved	100000009

Status	Value (example)
In Execution	100000010
On Hold	100000011
Execution Completed	100000012
Closed	100000013
Cancelled	100000014

The Dataverse trigger, explained

Most flows start with **"When a row is added, modified or deleted"** (Dataverse). Its settings:

- **Change type** – Added, Modified or Deleted. Here mostly Modified.
- **Table name** – TV Requests.
- **Scope** – Organization = react to changes made by anyone.
- **Select columns** – react only when these columns change. With `tvr_status`, editing the title does not start the flow.
- **Filter rows** – an extra condition. `tvr_status eq 100000001` = only when the new status is Submitted.

The common building block: "send a template email and log it"

Many flows send emails the same way. You can copy these steps, or make them one child flow **F00 - Send Notification** with inputs TemplateCode, TVRequestId, ToEmails and ExtraComments.

1. **List rows** – table Email Templates, Filter rows: `tvr_templatecode eq 'REQ_SUBMITTED'` and `tvr_isactive eq true`, Row count = 1. This fetches the template.
2. **Compose – Subject:**

```
replace(replace(first(outputs('List_template'))['body/value'])?
['tvr_subject'],
  '{TVID}', triggerOutputs()['body/tvr_tvid']),
  '{Title}', triggerOutputs()['body/tvr_title'])
```

Read from the inside: `first(...)` takes the one template row, `?['tvr_subject']` takes its subject, the inner replace swaps `{TVID}` for the real TVID, and the outer replace swaps `{Title}` for the real title.

3. **Compose - Body:** the same replace for {TVID}, {Title}, {RequesterName}, {Status} and {Comments}. The link is built as `concat(parameters('App URL (tvr_AppUrl)'), '?TVID=', triggerOutputs()['body/tvr_tvid'])`.
4. **Send an email (V2)** (Office 365 Outlook) - To, Subject, Body (HTML). Tip: use "Send an email from a shared mailbox (V2)" with tvrms-noreply@lam.com, so all emails come from one address.
5. **Add a new row - Notification Logs:** Subject, Template Code, Sent To, Sent On = `utcNow()`, Result = Sent, TV Request = `tvr_tvrequests(<id>)`. To fill a lookup in a flow, you write the table's plural name plus the row's ID in brackets.
6. If step 4 fails, add a Notification Log row with Result = Failed and the error text. Use **Configure run after → has failed** on this step, so it runs only when the email step failed.

If you build F00 as a child flow, its trigger has no request row. So first add **Get a row by ID** (TV Requests, TVRequestId) and use that output instead of `triggerOutputs()`.

Sample email (REQ_SUBMITTED):

Subject: [TVRMS] TV-2026-00015 submitted - Plasma etch uniformity test on 300mm wafers

Dear Priya Sharma,

Your Test Vehicle request TV-2026-00015 has been submitted successfully and is now with the APPG team for review.

Request type : New

Required by : 15-Dec-2026

Status : Submitted

Open request : <https://apps.powerapps.com/play/e/.../a/...?TVID=TV-2026-00015>

Regards,

TV Request Management System (automatic email - please do not reply)

F01 - On Request Submitted

Purpose: create the SharePoint workspace, thank the requester, and tell APPG.

Trigger: Dataverse "When a row is added, modified or deleted" - Change type = Modified, Table = TV Requests, Scope = Organization, Select columns = `tvr_status`, Filter rows = `tvr_status eq 100000001` (Submitted).

Steps:

1. **Scope - Try** (a box that holds the main steps).

2. **Initialize variable** FolderList (Array): ["01-Request", "02-Technical Documents", "03-Feasibility & Quotes", "04-Funding Approval", "05-Execution", "06-Final Report & Closure"] .
3. **Apply to each** item of FolderList → SharePoint **Create new folder**. Site = env var tvr_SPSiteUrl, List = tvr_SPLibrary, Folder path = @{triggerOutputs()?['body/tvr_tvid']}/@{items('Apply_to_each')} . Example path: TV-2026-00015/01-Request.
4. **Update a row – TV Requests:** SharePoint Folder URL =
concat(parameters('SharePoint Site URL (tvr_SPSiteUrl)'), '/',
parameters('SharePoint Library (tvr_SPLibrary)'), '/', triggerOutputs()?['body/tvr_tvid']) .
5. **Get a row by ID – Users:** the requester, using `_tvr_requester_value` (the hidden ID inside the Requester lookup) → email and full name.
6. (Optional, stricter security) SharePoint **Send an HTTP request** to break inheritance on the TVID folder and give Edit to the requester (section 9).
7. Send template email **REQ_SUBMITTED** → the requester.
8. Send template email **NEW_REQUEST_APPG** → the address in env var tvr_APPGEmail.
9. **Scope – Catch** (Configure run after: Try has failed or timed out) → the F08 error steps.

Test: submit TV-2026-00015 → the folder TV Requests/TV-2026-00015 exists with 6 sub-folders, Priya gets an email, the APPG mailbox gets an email, and 2 Notification Log rows exist.

F02 – Status Change Notifier

Purpose: one flow that sends the right email for every status change.

Trigger: Dataverse, Modified, TV Requests, Select columns = `tvr_status` , no filter (it reacts to every status).

Steps:

1. Get the emails of the requester, the APPG owner and the cost center owner (Get a row by ID, three times). Set **Configure run after** so the flow continues when a lookup is empty – for example, there is no APPG owner yet in early statuses.
2. Get the latest comment: **List rows** Request Comments, filter on this request, sort `createdon desc` , Row count 1. Its text becomes {Comments}.
3. **Switch** on `triggerOutputs()?['body/tvr_status']` :

New status	Template	To	CC
Under APPG Review	REVIEW_STARTED	Requester	-

New status	Template	To	CC
More Info Required	MORE_INFO	Requester	APPG owner
Rejected	REJECTED	Requester	APPG team
Internal Alignment 1 / 2	ALIGNMENT_UPDATE	Requester, stakeholders	APPG owner
Funding Approved	FUNDING_APPROVED	Requester, APPG owner	CCO
Funding Rejected	FUNDING_REJECTED	Requester, APPG owner	CCO
In Execution / On Hold	STATUS_UPDATE	Requester, stakeholders	-
Execution Completed	STATUS_UPDATE	Requester	-
Cancelled	CANCELLED	Requester	APPG owner
Submitted, Pending Funding Approval, Closed	(do nothing - F01, F04 and F07 handle these)		

4. Send the template email and log it (the building block above).

Example: David clicks Request More Info with the comment "Please attach the wafer map." F02 sees status More Info Required and sends MORE_INFO to Priya, with the comment inside the email and David in CC.

Three practical details:

- A Switch case holds one value. "Internal Alignment 1 / 2" therefore needs two cases (100000005 and 100000006) with the same steps; the same for "In Execution / On Hold".
- The guide does not say how F02 finds the "stakeholders". They are the members of the request's access team. Reading them needs an extra step, or you can keep a simple "Stakeholder Emails" text column on the request.
- **F02b - exception alert** is a second small flow: Risk Issues, "When a row is added", Filter rows = `tvr_severity eq <the number for High>` → email the APPG owner and the requester with template EXCEPTION_RAISED.

15. Doc 08 (part 2) – Flows F04 to F10, and how to test them

These flows handle the money approval (F04), the daily reminders (F05), the PDF (F06, F07), error handling (F08), file upload (F09) and adding stakeholders (F10). Each is explained below with its trigger, its steps and a sample run.

F04 – Funding Approval Routing

Trigger: TV Requests modified, Select columns = `tvr_status`, Filter rows = `tvr_status eq 100000007` (Pending Funding Approval).

Steps:

1. **Get a row by ID – Cost Centers** (using `_tvr_costcenter_value`) → the Owner and the Owner's Manager.
2. **Get a row by ID – Users** → the CCO's email and the manager's email.
3. **Get a row by ID – Feasibility Options** (the selected option) → the quoted cost and the institution.
4. **List rows – App Settings**, filter `tvr_name eq 'HighValueFundingLimit'` → the limit (50000).
5. **Add the CCO to the request's access team** (AddUserToRecordTeam), so Maria can open the request (section 10). The guide calls it an "unbound action". In the flow designer you will normally find it under **Perform a bound action**: Table = Users, Row ID = the CCO's user ID, Action = AddUserToRecordTeam, with the request as Record and "TVR Request Team" as TeamTemplate.
6. **Add a new row – Funding Approvals**: Request, Cost Center, Approver = CCO, Approval Level = 1, Requested Amount = quoted cost, Decision = Pending, Requested On = `utcNow()`, Name = "FA-" + TVID + "-1".
7. **Start and wait for an approval** (Approvals connector):
 - Type: **Approve/Reject – First to respond** (with one approver, the first answer is the only answer).
 - Title: `Funding approval: @{TVID} – USD @{formatNumber(QuotedCost, 'N0')}`. 'N0' formats 39500 as "39,500".
 - Assigned to: the CCO's email.
 - Details (markdown): request, requester, cost center, selected option, justification.
 - Item link: the app deep link `?TVID=...`.
8. **Update a row – Funding Approval**: Decision = the outcome; Comments = `first(body('Start_and_wait_for_an_approval')?['responses'])?['comments']` (the comment Maria typed); Decided On = `utcNow()`.

9. **Condition:** outcome = Approve **and** amount > limit? If yes, repeat steps 6–8 with Level = 2 and Approver = the Owner's Manager.

10. **Condition:** final outcome = Approve?

- Yes: update the TV Request → Status = Funding Approved, Approved Amount = quoted cost, Status Changed On = `utcNow()` ; add a Status History row (Changed By = the approver).
- No: update the TV Request → Status = Funding Rejected, Reason = the approver's comments; add a Status History row.

11. Because the status changed, F02 then sends FUNDING_APPROVED or FUNDING_REJECTED by itself.

"Start and wait" means the flow pauses until Maria clicks Approve or Reject – in Outlook, in Teams or in the Approvals app.

Two sample runs:

Quoted cost	Limit	Who approves	Result
USD 39,500	50,000	Maria only (level 1)	Funding Approved
USD 62,000	50,000	Maria (level 1), then James Wilson (level 2)	Funding Approved, with 2 Funding Approval rows

Time limit: one flow run can wait at most 30 days. While the approval is pending, F05 sends reminders to the CCO. If 30 days pass, the run ends and F05 flags the request to the APPG owner.

F05 – Daily Reminders, Escalations and Auto-Cancellation

Trigger: Recurrence – every weekday at 08:00, LAM's local time zone (Frequency = Week, days Monday to Friday, hour 8).

Steps:

1. Read the settings: ReminderAfterDays (3), EscalationAfterDays (5), AutoCancelAfterDays (30).
2. **List rows – TV Requests** that wait for someone's action and have not moved for at least 3 days:

```
(tvr_status eq 100000001 or tvr_status eq 100000002 or tvr_status eq
100000003
or tvr_status eq 100000005 or tvr_status eq 100000006 or tvr_status eq
100000007)
and tvr_statuschangedon le @{addDays(utcNow(), mul(-1,
int(variables('ReminderDays'))))}
```

The statuses are Submitted, Under APPG Review, More Info Required, Internal Alignment 1 and 2, and Pending Funding Approval. **le** means "less than or equal": the status changed 3 or more days ago.

3. Apply to each request:

- Days waiting: `div(sub(ticks(utcNow()), ticks(items('Apply_to_each')?['tvr_statuschangedon'])), 8640000000000)`. A tick is a tiny time unit; one day is 864,000,000,000 ticks, so the difference divided by that is the number of whole days.
- Who must act: More Info Required → the requester; Pending Funding Approval → the CCO; all others → the APPG owner (or the APPG team if there is no owner yet).
- A - auto-cancel:** status = More Info Required and days ≥ 30 → Status = Cancelled, Reason = "Auto-cancelled: no response for 30 days", add a Status History row (Changed By = system). F02 then sends CANCELLED.
- B - escalate:** days ≥ 5 → Office 365 Users **Get manager (V2)** of the person who must act → send ESCALATION to the manager, with that person in CC.
- Otherwise - remind:** send REMINDER to the person who must act.

4. **List rows - Milestones** whose Planned End has passed and Status is not Completed → "Milestone overdue" reminder to the APPG owner.

5. **List rows - TV Requests** In Execution whose Expected Completion Date has passed → email the APPG owner.

Sample run on 12-Oct-2026:

TVID	Status	Days waiting	Action
TV-2026-00018	Submitted	3	Reminder to the APPG team (no owner yet)
TV-2026-00016	More Info Required	6	Escalation to Priya's manager
TV-2026-00009	More Info Required	31	Auto-cancelled

TVID	Status	Days waiting	Action
TV-2026-00012	Pending Funding Approval	4	Reminder to the CCO

Three things to decide or adjust:

- **Calendar or working days?** The guide assumes calendar days. For working days, keep a Holiday table and count only Monday to Friday, non-holiday days.
- **How days are counted.** The sample table counts calendar dates ($12 - 9 = 3$). The ticks formula counts full 24-hour days: TV-2026-00018 changed on 9-Oct at 15:00, so at 08:00 on 12-Oct only 2 full days have passed and no reminder goes out. To match the table, apply `startOfDay()` to both times before subtracting, in the formula and in the filter.
- **Repeat emails.** As written, from day 5 the manager gets an escalation every weekday until someone acts. Ask whether one escalation is enough.

F06 - Generate PDF and Store

The guide uses two flows, so the PDF logic exists only once:

- **F06a - Build Request PDF** (a child flow): trigger "Manually trigger a flow" with input TVID.
- **F06 - Export PDF** (called by the app): trigger Power Apps (V2) with input TVID → **Run a Child Flow F06a** → **Respond to a PowerApp or flow** with `fileurl`.

F06a steps:

1. **List rows - TV Requests** where `tvr_tvaid eq '{@triggerBody()['text']}` (expand the requester and the cost center).
2. **List rows** - the Feasibility Options, Funding Approvals and Milestones of this request.
3. **Create HTML table** (a Data Operation) for the options, the approvals and the milestones.
4. **Compose** the full HTML page:

```
<html><body style="font-family: Segoe UI">
<h1>TV Request Summary - TV-2026-00015</h1>
<p><b>Title:</b> Plasma etch uniformity test ... <b>Status:</b> Closed</p>
<h2>Feasibility Options</h2> @{body('Create_HTML_table_options')}
<h2>Funding Approvals</h2> @{body('Create_HTML_table_approvals')}
<h2>Milestones</h2> @{body('Create_HTML_table_milestones')}
<h2>Closure</h2><p>Outcome: ... Lessons learned: ...</p>
<p>Generated on @{formatDateTime(utcNow(), 'dd-MMM-yyyy HH:mm')} UTC</p>
</body></html>
```

5. **OneDrive for Business - Create file** /TVRMS-Temp/{TVID}.html with that HTML.
6. **OneDrive - Convert file** (by Id) → target type PDF. OneDrive is used because it can turn HTML into PDF; there is no simpler standard action for this.
7. **SharePoint - Create file** in /@{TVID}/06-Final Report & Closure (or 01-Request if the request is not closed). Name:
`@{TVID}_Summary_{formatDateTime(utcNow(), 'yyyyMMdd')}.pdf` , for example TV-2026-00015_Summary_20270305.pdf.
8. **Add a Request Document row** (type Approval Evidence or Final Report).
9. **OneDrive - Delete file** (the temporary HTML).
10. **Respond to a PowerApp or flow:** fileurl = the SharePoint file link.

Option: if LAM has a Word template, "Populate a Microsoft Word template" (a premium action) can replace the HTML step.

F07 - On Request Closed (completion notification)

Trigger: TV Requests modified, Select columns = `tvr_status` , Filter rows = `tvr_status eq 100000013` (Closed).

1. **Run a Child Flow F06a** → the closure summary PDF is saved in 06-Final Report & Closure.
2. Find the final report: **List rows - Request Documents**, Document Type = Final Report, top 1.
3. Send template **COMPLETED** to the requester, the APPG owner, the CCO and the stakeholders, with links to the final report and the summary PDF (slide 4's "Report Available Notification").
4. If Request Type = Recurring → send **NEXT_OCCURRENCE** to the requester: "Your next Quarterly TV request is due. Use 'Repeat' to create it."

Note: the COMPLETED template uses {ReportLink} and {PdfLink}, and the REMINDER and ESCALATION templates use {Days} and {ActionOwner}. The common email block replaces only 6 placeholders, so add replace steps for these 4 in F05 and F07.

F08 - The error-handling pattern (exception remediation)

Every flow keeps its steps in **Scope - Try**. After it comes **Scope - Catch**, set to run only when Try **has failed** or **has timed out** (Configure run after). The Catch scope does 3 things:

1. **Add a new row - Error Logs:**
 - Flow Name: `@{workflow()}['tags']['flowDisplayName']`

- Error: `@{string(result('Try'))}` – the results of every step inside Try, including the error
 - Run URL: `concat('https://make.powerautomate.com/environments/', workflow()?['tags']?['environmentName'], '/flows/', workflow()?['name'], '/runs/', workflow()?['run']?['name'])` – a direct link to this failed run
 - TV Request: the ID of the trigger row
2. **Email** the address in env var `tvr_AdminEmail`: "TVRMS flow failed – {Flow} – {TVID}" plus the Run URL.
 3. **Terminate** with Status = Failed. Without this, the run would show green ("Succeeded") because Catch handled the error, and the failure would be hidden.

`workflow()` gives facts about the running flow (its name, environment and run ID), which is how the link is built.

The admin then opens **Monitoring → Open Errors** in the admin app, fixes the cause, resubmits the failed run from the run history, and sets the Error Log Status to Fixed.

Also set the **Retry policy** on SharePoint and Outlook actions (the action's Settings): Exponential, 4 retries. Short network hiccups then fix themselves.

F09 – Upload Document (called from the app)

Trigger: Power Apps (V2) with 4 inputs:

Input	Type	Sample
TVID	Text	TV-2026-00015
DocType	Text	Technical Specification
Folder	Text	02-Technical Documents
file	File	Etch_Test_Spec_v1.pdf

Inside the flow, text inputs get automatic names in the order you add them: `text` (TVID), `text_1` (DocType), `text_2` (Folder).

1. **SharePoint – Create file:** Folder path = `/@{parameters('SharePoint Library (tvr_SPLibrary)')}/@{triggerBody()['text']}/@{triggerBody()['text_2']}`, File name = `@{triggerBody()?['file']?['name']}`, File content = `@{triggerBody()?['file']?['contentBytes']}`. If the folder does not exist yet, SharePoint creates it. The same file name again becomes a new version.
2. **SharePoint – Update file properties** (Id = `outputs('Create_file')?['body/ItemId']`): TVID, DocumentType, Phase.

3. **List rows – TV Requests** where `tvr_tvuid eq '@{triggerBody()['text']}'`.

4. **Add a new row – Request Documents:** File Name, Document Type, Folder, SharePoint URL, TV Request.

5. **Respond to a PowerApp or flow:** fileurl.

Two small details: Phase is not an input, so take it from the request row (step 3) and do step 3 before step 2. And Document Type in the Request Documents table is a choice (a number), so turn the text "Technical Specification" into its number first, for example with a small Switch.

F10 – Add Stakeholder (optional, from the app)

Power Apps (V2) trigger with TVID and StakeholderEmail → **Get user** → **AddUserToRecordTeam** (template TVR Request Team) → email **STAKEHOLDER_ADDED**.

Example: David adds Tom Baker to TV-2026-00015. Tom can now open that request read-only and gets status emails.

The template STAKEHOLDER_ADDED is not among the 15 sample templates in email_templates.csv, so add it as a 16th row.

Flow testing checklist

Flow	How to test	Pass when
F01	Submit a request	6 folders created, 2 emails sent, folder URL saved
F02	Change the status to More Info Required	Requester gets MORE_INFO with the comment
F04	Send for funding with amount 39,500	CCO gets the approval in Outlook / Teams; decision saved
F04	Amount 62,000	Two approvals, one after the other
F05	On a More Info request, set Status Changed On to 31 days ago; run the flow by hand	Request auto-cancelled
F06	Click Export PDF	PDF opens; the file is in SharePoint
F07	Close a request	Completion email with both links
F08	Temporarily put a wrong library name in the env variable, then submit	An Error Log row and an admin email

Flow	How to test	Pass when
F09	Upload 2 files	Both in SharePoint, plus 2 Request Document rows

16. Doc 09 – Power BI dashboard and reports

The Power BI report reads the Dataverse tables, adds about 20 calculations (DAX measures), and shows 6 pages that each answer one management question. It refreshes 4 times a day. Slide 5 lists the topics: request status, cycle time and aging, cost tracking, utilization and performance, exception analysis, executive reporting.

1. Connect to the data

1. Open **Power BI Desktop** → **Get data** → **Dataverse** → sign in → choose the environment (LAM-TVRMS-DEV while building, LAM-TVRMS-PROD at the end).
2. Select 11 tables: tvr_tvrequest, tvr_feasibilityoption, tvr_fundingapproval, tvr_milestone, tvr_riskissue, tvr_costentry, tvr_statushistory, tvr_department, tvr_costcenter, tvr_location, tvr_researchinstitution.
3. Mode = **Import**. Import keeps a copy of the data inside the report, so it is fast; it refreshes 4 to 8 times a day. **DirectQuery** reads live data on every click but is slower; use it only if live numbers are a must.
4. In **Power Query**: keep only the columns you need, give them friendly names, and use the label columns for choices. The connector gives a "...name" column next to each choice (for example the label "Submitted" next to the number 100000001).
5. Add a **Date table** (DAX):

```
Date = ADDCOLUMNS(CALENDAR(DATE(2026,1,1), DATE(2030,12,31)),
  "Year", YEAR([Date]), "Month", FORMAT([Date],"MMM yyyy"),
  "MonthNo", YEAR([Date])*100+MONTH([Date]), "Quarter", "Q" &
  QUARTER([Date]))
```

CALENDAR makes one row per day from 2026 to 2030. ADDCOLUMNS adds Year (2026), Month ("Oct 2026"), MonthNo (202610, used to sort the months in the right order) and Quarter ("Q4").

6. Relationships (Model view):

- TV Request → Department, Cost Center, Location (many-to-one).
- Feasibility, Funding, Milestone, Risk, Cost and Status History → TV Request (many-to-one).

- `Date[Date] → TV Request[Submitted On (date)]`. Submitted On holds a date and a time, so first add a date-only copy of it.

2. The DAX measures, one by one

A few DAX words first: **CALCULATE** = count or sum with an extra filter; **DIVIDE** = a safe division that does not fail when dividing by zero; **AVERAGEX** = an average of a value worked out row by row; **VAR / RETURN** = keep a temporary value and then use it; **SWITCH(TRUE(), ...)** = the first true condition wins.

Put all measures in one empty table named "_Measures" so they are easy to find. The last column shows the value with the sample CSV files (my calculation where the guide gives none).

Measure	Formula	Simple meaning	Sample value
Total Requests	<code>COUNTROWS('TV Request')</code>	Count every request	8
Open Requests	<code>CALCULATE([Total Requests], NOT 'TV Request'[Request Status] IN {"Closed", "Cancelled", "Rejected", "Funding Rejected"})</code>	Requests not in a final status	5
Closed Requests	<code>CALCULATE([Total Requests], 'TV Request'[Request Status] = "Closed")</code>	Finished requests	1
Rejected Requests	<code>CALCULATE([Total Requests], 'TV Request'[Request Status] IN {"Rejected", "Funding Rejected"})</code>	Stopped by APPG or by the CCO	1
Approval Rate %	<code>DIVIDE(CALCULATE([Total Requests], NOT ISBLANK('TV Request'[Approved Amount])), CALCULATE([Total Requests], 'TV Request'[Request Status] IN {"Funding Approved", "Funding Rejected", "In Execution", "On Hold", "Execution Completed", "Closed"}))</code>	Of the requests that got a funding answer, the share that got money	100% (3 of 3)
Avg Cycle Time (days)	<code>AVERAGEX(FILTER('TV Request', NOT ISBLANK('TV Request'[Closed On])), DATEDIFF('TV Request'[Submitted On], 'TV Request'[Closed On], DAY))</code>	Average days from submit to close, closed requests only	115

Measure	Formula	Simple meaning	Sample value
Age (days)	<code>DATEDIFF(MAX('TV Request'[Submitted On]), TODAY(), DAY)</code>	Days since submit; use it in a table, one row per request	depends on today
Days In Current Status	<code>DATEDIFF(MAX('TV Request'[Status Changed On]), NOW(), DAY)</code>	Days since the last status change	depends on today
Approved Budget	<code>SUM('TV Request'[Approved Amount])</code>	Money approved	82,500
Actual Cost	<code>SUM('Cost Entry'[Amount])</code>	Money spent	57,500
Cost Variance	<code>[Approved Budget] - [Actual Cost]</code>	Money left	25,000
Budget Utilization %	<code>DIVIDE([Actual Cost], [Approved Budget])</code>	Share of the approved money spent	70%
On-Time Completion %	<code>DIVIDE(CALCULATE([Closed Requests], FILTER('TV Request', 'TV Request'[Closed On] <= 'TV Request'[Required By Date])), [Closed Requests])</code>	Share of closed requests finished by their Required By date	100%
Open Issues	<code>CALCULATE(COUNTROWS('Risk Issue'), 'Risk Issue'[Status] <> "Resolved")</code>	Problems not yet solved	0 (no sample rows)
High Severity Issues	<code>CALCULATE([Open Issues], 'Risk Issue'[Severity] = "High")</code>	Open problems marked High	0

Measure	Formula	Simple meaning	Sample value
Overdue Milestones	<code>CALCULATE(COUNTROWS('Milestone'), 'Milestone'[Planned End] < TODAY(), 'Milestone'[Status] <> "Completed")</code>	Steps past their planned end and not done	depends on today

How the sample values come out:

- Approved Budget: 15,000 (TV-2026-00001) + 28,000 (TV-2026-00003) + 39,500 (TV-2026-00015) = 82,500.
- Actual Cost: 18,000 + 5,300 + 14,000 + 6,000 + 14,200 = 57,500 (all 5 rows of cost_entries.csv).
- Avg Cycle Time: only TV-2026-00001 is closed: 2-Mar-2026 to 25-Jun-2026 = 115 days. It was needed by 30-Jun, so On-Time = 100%.
- The guide's KPI "Rejected / Cancelled = 2" counts TV-2026-00011 (Rejected) and TV-2026-00009 (Cancelled). The Rejected Requests measure alone gives 1, so make a separate measure that also includes Cancelled.
- The sample data has no Risk / Issue file, so the issue visuals stay empty until you add test rows.

Aging Bucket (a calculated column on TV Request – it gives each request a label):

```
Aging Bucket =
VAR d = DATEDIFF('TV Request'[Submitted On], IF(ISBLANK('TV Request'[Closed On]), TODAY(), 'TV Request'[Closed On]), DAY)
RETURN SWITCH(TRUE(), d <= 15, "0-15 days", d <= 30, "16-30 days", d <= 60, "31-60 days", "60+ days")
```

d = days from submit to close (or to today if still open). Then the first matching band wins. Example: TV-2026-00015 took 144 days → "60+ days".

Hours In Status (a calculated column on Status History – where time is lost):

```
Hours In Status =
VAR thisReq = 'Status History'[TV Request]
VAR thisTime = 'Status History'[Changed On]
VAR nextTime = CALCULATE(MIN('Status History'[Changed On]),
    FILTER(ALL('Status History'), 'Status History'[TV Request] = thisReq &&
        'Status History'[Changed On] > thisTime))
RETURN DATEDIFF(thisTime, IF(ISBLANK(nextTime), NOW(), nextTime), HOUR)
```

For each history row: find the next status change of the same request; the hours between the two = the time spent in this status. If there is no next change, count up to now.

Example: TV-2026-00015 entered More Info Required on 13-Oct at 09:40 and left on 15-Oct at 14:20 → about 53 hours (about 2 days). Averaging Hours In Status by To Status gives the **bottleneck chart**: the statuses where requests wait longest.

3. The 6 report pages

Page	Visuals	Answers the question
1. Executive Summary	KPI cards (Total, Open, Closed, Avg Cycle Time, Approved Budget, Utilization %), requests by month (column chart), status funnel	"How are we doing overall?"
2. Request Status	Bar by status, donut by phase, table of open requests with TVID, owner and days in status, slicers (Department, Request Type, Priority, Date)	"Where is every request now?"
3. Cycle Time & Aging	Aging bucket bar, average cycle time by department, hours in each status (bottleneck), on-time %	"Where do we lose time?"
4. Cost Tracking	Approved vs actual per request (clustered bar), cost by category (pie), by cost center (matrix), variance	"Are we within budget?"
5. Utilization & Performance	Requests per research institution, average quoted cost and weeks per institution, budget utilization %, reuse recommendation counts	"Which institutions perform best?"
6. Exception Analysis	Open issues by severity, overdue milestones, auto-cancelled and rejected requests with reasons, failed notifications	"What is going wrong?"

A slicer is a clickable filter on the page, for example "Department = Metrology".

4. Security in Power BI

The Dataverse connector shows only what the person who **refreshes** the report is allowed to see. With Import mode the service account refreshes, so the report contains every request, and everyone who opens it sees everything. Therefore:

- Share the Executive report only with APPG and management (SG-TVRMS-APPG and a leadership group).
- If requesters need a "My requests" report, add **Row-Level Security**: a role "Requester" with the filter `[Requester Email] = USERPRINCIPALNAME()` on TV Request. `USERPRINCIPALNAME()` is the email of the person viewing the report, so Priya sees only her rows. The model needs a Requester Email column for this.

5. Publish

1. Publish to a workspace named **LAM TVRMS** in the Power BI service.
2. In the dataset (semantic model) settings: credentials = OAuth with the service account; **Scheduled refresh** 4 times a day.
3. Create an **App** in the workspace and share it with the right groups.
4. Optional: show the report inside the Canvas app (Power BI tile control) or as a dashboard page in the model-driven app.

17. Doc 10 – Testing and UAT

Testing happens in 5 layers, from one screen alone up to a quick check in production. The guide gives 7 test users (one per role) and 33 test cases (TC-01 to TC-33) that cover every phase, the automation, security, PDF export, audit, settings, browsers and reports. UAT runs for 3 to 5 days in TEST and ends with a sign-off email from the business.

Slide 6 says: validate the functionality, UAT with limited users, add UAT feedback, fix issues. Slide 8 says: UAT is done by business users.

1. The 5 types of testing

Type	What it means	Who	Where	When
Unit testing	Test one screen or one flow on its own	Developer	DEV	During each sprint
System / integration testing	Test the whole process from start to end	Developer + QA	TEST	End of each sprint

Type	What it means	Who	Where	When
Security testing	Log in as each role and check what they can see and do	Developer	TEST	Sprint 2
UAT (User Acceptance Testing)	Business users confirm the system does what they need	3-5 business users (ASSUMPTION)	TEST	Weeks 6-7
Smoke test	A quick check that the main path works after deployment	Developer + 1 key user	PROD	Go-live day

2. Test users (create in TEST, one per role)

Test user	Role	Department / Cost center
test.requester1@lam.com (Priya)	TVR Requester	Process Engineering
test.requester2@lam.com (John)	TVR Requester	Metrology
test.appg@lam.com (David)	TVR APPG Reviewer	-
test.cco@lam.com (Maria)	TVR Cost Center Owner	CC-4100
test.ccomgr@lam.com (James)	Manager of Maria	-
test.stakeholder@lam.com (Tom)	TVR Stakeholder	-
test.admin@lam.com	TVR Admin	-

For escalations to work, set Maria as James's report and set a manager for Priya in Entra ID, because F05 uses "Get manager".

3. The 33 test cases (with sample data)

ID	Area	Scenario	Steps	Expected result
TC-01	Initiation	Mandatory fields	Priya clicks Submit with Title empty	Error "Please fill all fields marked with *"; nothing saved
TC-02	Initiation	Technical document mandatory	Fill all fields, no attachment, Submit	Error "attach at least one technical specification"

ID	Area	Scenario	Steps	Expected result
TC-03	Initiation	Past date	Required By = yesterday	Error about a future date
TC-04	Initiation	Happy path, New	"Plasma etch uniformity test", New, PE, Fremont, CC-4100, High, 15-Dec-2026, 42,000, attach Etch_Test_Spec_v1.pdf, Submit	TVID created, status Submitted, 6 folders, file in 02-Technical Documents, email to Priya, email to APPG
TC-05	Initiation	Repeat request	Type = Repeat, select TV-2026-00003	Fields pre-filled; Previous Request saved
TC-06	Initiation	Recurring without frequency	Type = Recurring, no frequency	Error, cannot submit
TC-07	Initiation	Save Draft	Fill Title only → Save Draft	Saved as Draft; no email; no folder
TC-08	APPG Review	Start review and assign owner	David opens the request, Start Review, owner = David	Under APPG Review; REVIEW_STARTED to Priya; history row
TC-09	APPG Review	More info	David comments "Please attach the wafer map" → Request More Info	More Info Required; Priya's email shows the comment
TC-10	APPG Review	More info without comment	Click Request More Info with an empty comment	Error; status unchanged
TC-11	APPG Review	Requester responds	Priya uploads Wafer_Map.xlsx, Resubmit	Under APPG Review; APPG notified
TC-12	APPG Review	Reject	David rejects with "Duplicate of TV-2026-00011"	Rejected; reason saved; email to Priya
TC-13	Alignment	Add 3 options	Options A, B, C (section 12)	3 rows; comparison visible
TC-14	Alignment	Select option	Select Option A	Only A selected; Estimated Budget = 39,500

ID	Area	Scenario	Steps	Expected result
TC-15	Alignment	Alignment 2 without option	Click Confirm Option with none selected	Button disabled or error
TC-16	Funding	Normal approval	Send for Funding (39,500) → Maria approves "OK from Q4 budget"	Funding row Approved; Funding Approved; Approved Amount 39,500; emails
TC-17	Funding	High value	Amount 62,000 → Maria approves → James approves	2 funding rows (L1, L2); Funding Approved
TC-18	Funding	Rejection	Maria rejects "No budget this quarter"	Funding Rejected; Priya gets the reason
TC-19	Execution	Milestones and progress	Add 4 milestones; set 2 to 100%	Overall Progress = 50%
TC-20	Execution	Cost tracking	Add costs 18,000 + 12,000	Actual 30,000; Utilization 76%
TC-21	Execution	High issue	Add issue "Lab tool down", severity High	EXCEPTION_RAISED email
TC-22	Closure	Close without final report	Click Close	Error "upload the final report first"
TC-23	Closure	Close	Fill outcome, lessons, reuse; upload Final_Report.pdf; Close	Closed; summary PDF in the 06 folder; completion email to all
TC-24	Automation	Reminder	A request Submitted 3 days ago → run F05	Reminder email to APPG
TC-25	Automation	Escalation	More Info Required for 6 days	Email to the requester's manager
TC-26	Automation	Auto-cancel	More Info Required for 31 days	Cancelled, reason "Auto-cancelled..."
TC-27	Security	Requester isolation	John opens the app	Cannot see Priya's requests
TC-28	Security	CCO view	Maria opens the link from the approval email	Can view TV-2026-00015 read-only

ID	Area	Scenario	Steps	Expected result
TC-29	Export	PDF	Click Export to PDF	PDF opens; saved in SharePoint
TC-30	Audit	History	Open Audit History in the admin app	Every field change with user and time
TC-31	Admin	Change a setting	ReminderAfterDays = 2	The next F05 run uses 2 days
TC-32	NFR	Browser	Open the app in Microsoft Edge and Chrome	Works the same
TC-33	Reports	Dashboard	Refresh Power BI after the tests	Counts match Dataverse

How to check the numbers: TC-19 = $(100 + 100 + 0 + 0) \div 4 = 50\%$. TC-20 = $30,000 \div 39,500 = 75.9\%$, shown as 76%.

How to fake old dates for TC-24 to TC-26: in TEST, open the request in the admin app and change Status Changed On to 3, 6 or 31 days ago, then run F05 by hand.

Four test cases need a small addition to the guide's design before they can pass:

- **TC-06** – the Submit formula has no Recurring check. Add one more condition: Request Type = Recurring and Frequency empty → error. Or make the Frequency card required when the type is Recurring.
- **TC-10** – the guide writes out the mandatory-reason check only for Reject. Copy that pattern to Request More Info.
- **TC-11** – F02 sends the "Under APPG Review" email only to the requester, so APPG is not told about a resubmit. Add a case: when the newest Status History row comes from More Info Required, email the APPG owner.
- **TC-20** – Actual Total Cost is a rollup and updates late (section 12). Show the sum calculated in the app, or wait for the recalculation before you check.

4. The UAT process

1. **Prepare.** Load master data and sample data into TEST. Export the test-case table to Excel and add 3 columns for the testers: Actual result, Pass/Fail, Comments.
2. **Kick-off meeting (30 minutes).** Show the app and explain how to log an issue.
3. **Testers work for 3 to 5 days.** They log each defect in a list (Excel, Azure DevOps or a SharePoint list):

Defect ID	Test case	Description	Severity	Status	Fixed in
D-001	TC-09	Email shows {Comments} instead of the comment	High	Fixed	Build 1.0.0.5
D-002	TC-19	Progress % not refreshing on the screen	Medium	Fixed	1.0.0.5
D-003	-	Want the Department dropdown sorted A-Z	Low (enhancement)	Done	1.0.0.6

"Fixed in" is the solution version that contains the fix (section 5 explains version numbers).

4. Fix → deploy again to TEST → test again.

5. UAT sign-off. The business owner sends a sign-off email. Slide 6 says key stakeholders must approve and sign off before deployment. Save that email in the project folder.

Severity rules:

Severity	Meaning	Action
Critical	Blocks the process	Must fix
High	Wrong result	Must fix
Medium	A workaround exists	Fix before go-live if possible
Low	Cosmetic (looks only)	Fix if time allows

A new idea from a tester is not a defect. It is a Change Request, because the price is fixed (slide 10).

18. Doc 11 – Deployment, go-live, hypercare and handover

The solution is built only in DEV and moves to TEST and then PROD as a locked ("managed") package. Some things are not inside the package and must be set up by hand in each environment. Go-live is in Week 7 with a checklist, hypercare (extra support) runs in Weeks 8–9, and then the app is handed to the support team with a runbook.

Slide 6: Deployment = stakeholders approve and sign off, go-live preparation, go-live.

Hypercare = post-rollout support and handover to the application support team.

Deliverables: signed-off Power Platform applications, and handover to support.

1. ALM – how the solution travels

ALM (Application Lifecycle Management) means moving changes safely from "build" to "live".

- **Unmanaged solution** – open and editable. It lives in DEV, where you build.
- **Managed solution** – a sealed, installed package. It lives in TEST and PROD. Nobody edits its parts there; a change comes as a new version from DEV.

```

DEV (unmanaged) —export managed—> TEST (managed) —same file—> PROD
(managed)
|
└─ export unmanaged + unpack —> Git repository (folder /solution)

```

Rule: never build directly in TEST or PROD.

Option A – Power Platform Pipelines (easiest, built in)

1. An admin installs Power Platform Pipelines in a "host" environment.
2. Create a pipeline "TVRMS Pipeline": source = DEV, Stage 1 = TEST, Stage 2 = PROD.
3. In DEV: Solutions → TVRMS → **Pipelines** → **Deploy here**. Fill the environment variable values for the target and pick the connections. You can require an approval before anything reaches PROD.

Option B – manual export and import

1. DEV → Solutions → TVRMS → **Publish all customizations**.
2. Raise the version, for example 1.0.0.5 → 1.0.1.0.
3. **Export → Managed** → you get TVRMS_1_0_1_0_managed.zip.
4. TEST → Solutions → **Import** → choose the zip → set the connection references (the service account's connections) and the environment variable values (the TEST SharePoint address and so on) → Import.
5. After testing, import the **same zip** into PROD. What was tested is exactly what goes live.

Keep the source in Git. The Power Platform CLI (pac) turns the solution into readable files that Git can track:

```
# one time: sign in to DEV
pac auth create --environment https://lam-tvrms-dev.crm.dynamics.com
# every time you finish a feature
pac solution export --name TVRMS --path ./out/TVRMS.zip --managed false
pac solution unpack --zipfile ./out/TVRMS.zip --folder ./solution --
packagetype Unmanaged
git add solution && git commit -m "TVRMS 1.0.1.0 - funding approval flow" &&
git push
```

1. `pac auth create` - sign in to the DEV environment once.
2. `pac solution export` - download the unmanaged solution as a zip.
3. `pac solution unpack` - unzip it into many small readable files.
4. `git add / commit / push` - save this version with a message. Later you can see who changed what, and go back if needed.

2. What is NOT inside the solution (set up in each environment)

A solution carries the design - tables, apps, flows, roles. It does not carry data rows, or things that live outside Dataverse.

Item	How to do it
Master data (departments, cost centers, ...)	Import the CSV files from LAM (slide 8: the customer provides them)
App Settings and Email Templates rows	Import app_settings.csv and email_templates.csv (or use the Configuration Migration tool)
SharePoint site, library, columns, versioning, retention label	Create by hand (section 9) or with a PnP script
Entra ID groups and Dataverse group teams	IT and the admin center (section 10)
Auditing settings	Admin center (section 5)
Power BI report	Publish from Desktop and change the data source to PROD
Sharing the apps with groups	Share after the import
Turning flows on	Check that every flow is On after the import

3. Go-live checklist (Week 7)

One week before:

- ☐ UAT sign-off received (email saved in the project folder)
- ☐ PROD environment ready; licences assigned to all users
- ☐ PROD SharePoint site, library and retention label ready
- ☐ Master data from LAM received and checked (no empty Cost Center Owner!)
- ☐ Email templates approved by the business
- ☐ Go-live announcement drafted by LAM (change management is out of scope)

Go-live day:

- ☐ Import the managed solution to PROD and note the version (for example 1.0.2.0)
- ☐ Set the environment variables and the connections (service account)
- ☐ Load master data, settings and templates
- ☐ Turn on all flows; check that their owner is the service account
- ☐ Share the Canvas app (users) and the model-driven app (admins)
- ☐ Publish Power BI, set the refresh, share the app
- ☐ Smoke test in PROD: create 1 test request → submit → check the folder and the email → cancel it and mark it "TEST – ignore"
- ☐ Send the go-live email with the app link

Why "no empty Cost Center Owner"? F04 sends the funding approval to that person. An empty owner means an approval with nobody to answer it.

Rollback plan: if a critical problem appears, turn off the flows, tell users to use email for a short time, fix the problem in DEV, and deploy again. A managed solution can be upgraded quickly.

4. Hypercare (Weeks 8–9, about 2 weeks)

Hypercare is a period of extra close support right after go-live.

Activity	Detail
Daily check	Flow run history (failed runs), the Error Log, the Failed Notifications view
Daily 15-minute call	With the APPG lead: issues and questions
Issue log	Same format as the UAT defects
Quick fixes	Fix in DEV → TEST → PROD, in small releases
Usage check	How many requests were submitted; any request stuck

5. Handover to the application support team (BAU)

BAU ("business as usual") is the team that supports the app after the project ends. Give them a handover pack:

1. This repository (all documents and the exported solution).
2. The architecture drawing and the data model (section 6 and 7).
3. The list of flows with their purpose, trigger and owner (sections 14 and 15).
4. The security roles and how to add users: "add the person to Entra group X" (section 10).
5. A **runbook** – the common support tasks:

Problem	Fix
A user cannot open the app	Check the licence and membership of SG-TVRMS-Users
A user cannot see a request	Check the role, the access team and the owner of the request
An email was not received	Admin app → Failed Notifications → read the error; check the flow run
A folder was not created	Error Log → resubmit the F01 run
A Cost Center Owner left the company	Admin app → Cost Center → change the owner; reassign the pending approval (cancel it and send again)
Change the reminder days	App Settings table
A flow connection expired	Fix the service account's connection; all flows use connection references, so one fix covers them all

6. **Credentials ownership:** LAM IT manages the service account. Never use a personal account.

7. **Knowledge transfer session:** 1–2 hours, recorded.

19. Doc 12 – The 9-week project plan

The plan follows slide 7: Weeks 1–2 plan and blueprint, Weeks 3–6 build in two 2-week sprints, Week 7 go-live, Weeks 8–9 hypercare and handover. There are 3 formal deliverables: the signed Solution Report (end of Week 2), the signed-off applications (Week 7), and the handover to support (Week 9).

What is a sprint? A fixed block of building time – here 2 weeks – that ends with a demo to the business. Each sprint delivers a working piece, not just code.

Week by week

Week	Phase (slide 7)	Main work	Output
W1	Plan for Success	Kick-off; workshops (process, data and documents, reports); send the question list (section 3); get access and licences; create the environments	Workshop notes and answers
W2	Plan for Success	Write the Solution Blueprint (data model, statuses, screens, flows, emails, KPIs, security); validate the plan; submit for approval	Deliverable: Solution Report (signed off)
W3	Sprint 1	Dataverse tables, choices, relationships, security roles; master data import; SharePoint site; app shell + Home + New Request wizard; F09 upload; F01 submit	Request intake works
W4	Sprint 1	Request Detail, APPG Queue, review actions (more info, reject, assign), comments, documents; F02 notifications; email templates; Sprint 1 demo	Demo 1 to the business
W5	Sprint 2	Feasibility screen, Internal Alignment 1 and 2, Funding screen, F04 approval routing, access team; Execution screen (milestones, risks, costs)	Full process up to execution
W6	Sprint 2	Closure, F06 PDF, F07 completion, F05 reminders / escalations / auto-cancel, F08 error handling, admin app, Power BI report; system test; UAT starts; Sprint 2 demo	The UAT build

Week	Phase (slide 7)	Main work	Output
W7	Deploy	UAT fixes, UAT sign-off, PROD setup, go-live	Deliverable: Signed-off Power Platform Applications
W8	Hypercare	Post-go-live support and fixes	Issue log
W9	Hypercare	Support, knowledge transfer, handover to the BAU team	Deliverable: Handover to Support

On slide 7, the two triangles mark the end of Sprint 1 (end of W4) and Sprint 2 (end of W6), and the star marks go-live (end of W7).

The sprint backlog (user stories)

Three words first:

- **User story** – one small need, written as "As a <role> I want <thing> so that <benefit>".
- **Acceptance criteria** – the check that proves the story is done.
- **Story points** – a relative size: 2 = small, 8 = big. They compare effort; they are not hours.

Sprint 1 (W3-W4) – 42 points

ID	User story	Acceptance criteria	Points
US-01	As an admin I want master data tables so that forms have dropdowns	Departments, cost centers, locations, research institutions created and imported	3
US-02	As a requester I want to create a TV request (New / Repeat / One-time / Recurring)	3-step wizard; Repeat copies data; TVID generated automatically	8
US-03	As a requester I want the system to check mandatory fields and the technical document	TC-01 to TC-03 and TC-06 pass	3
US-04	As a requester I want to upload documents	Files go to the TVID folder; a metadata row is created	5
US-05	As the business I want a TVID workspace created automatically	6 folders on submit	3
US-06	As a requester I want an acknowledgement email	Email within 2 minutes, with a link	2

ID	User story	Acceptance criteria	Points
US-07	As APPG I want a queue of requests and to assign an owner	Queue sorted by waiting days	5
US-08	As APPG I want to request more information or reject, with comments	Status + email + history	5
US-09	As a requester I want to see my requests and their status	Home screen counters + list	3
US-10	As the business I want role-based access	TC-27 passes	5

Sprint 2 (W5-W6) – 50 points

ID	User story	Acceptance criteria	Points
US-11	As APPG I want to compare feasibility options	Add, compare and select options	5
US-12	As APPG I want to run Internal Alignment 1 and 2	Status moves; alignment emails	3
US-13	As APPG I want funding approval routed to the CCO	TC-16 to TC-18 pass	8
US-14	As APPG I want to track milestones, risks and costs	TC-19 to TC-21 pass	8
US-15	As APPG I want to close a request with outcome and lessons learned	TC-22 and TC-23 pass	5
US-16	As the business I want reminders, escalations and auto-cancel	TC-24 to TC-26 pass	5
US-17	As a user I want to export a request to PDF	TC-29 passes	3
US-18	As an admin I want an admin app for master data and logs	Model-driven app with views	3
US-19	As management I want a Power BI dashboard	6 pages; numbers match	8
US-20	As support I want failed flows logged	Error Log row + email	2

Sprint 2 is the heavier sprint (50 points against 42), and it also starts UAT. If Sprint 1 slips, Sprint 2 is where the pressure lands, so protect Weeks 3–4.

The team (ASSUMPTION - typical for this size)

Role	Time on the project
Project Manager / Business Analyst	50%
Power Platform Developer (apps + flows)	100%
Power Platform Developer / Power BI	100% (W3-W7)
QA / Tester	50% (W4-W7)

Risks to watch

Risk	Impact	Action
Late master data from LAM	Cannot test properly	Use the sample data; ask for the real data in W1
Licences not ready	Blocks building and testing	Raise it on Day 1 (slide 8 dependency)
Requirements change after the blueprint	Delay and extra cost	Use the Change Request process
Experts (SMEs) not available for workshops or UAT	Delay	Book their calendars in W1

20. Doc 13 – One request from start to finish (TV-2026-00015)

This story ties every chapter together. Priya Sharma, a process engineer in Fremont, needs a plasma etch uniformity test because Customer X reported 4% non-uniformity. Her request TV-2026-00015 goes through all 6 phases in about 5 months, from 12-Oct-2026 to 5-Mar-2027, and the system does most of the paperwork by itself.

Phase 1 – Request Initiation (12-Oct-2026)

Priya opens the app and clicks New Request:

Field	Value
Title	Plasma etch uniformity test on 300mm wafers
Request Type	New
Department	Process Engineering
Location	Fremont, CA
Cost Center	CC-4100 Etch R&D
Priority	High
Required By	15-Dec-2026
Estimated Budget	USD 42,000
Business Justification	Customer X reports 4% within-wafer non-uniformity on 3nm etch. Need to qualify new recipe.
Technical Description	25 wafers, 3 recipes, CD-SEM measurement at 49 points per wafer.
Expected Outcome	Uniformity below 2%
Attachment	Etch_Test_Spec_v1.pdf

She clicks **Submit**. The system then:

1. Checks the fields and the attachment, and saves the record → TVID **TV-2026-00015** (section 12, blocks 1-2).
2. Flow F09 uploads the PDF to TV Requests/TV-2026-00015/02-Technical Documents (section 15).
3. Sets the status to Submitted and writes the history row "Draft → Submitted".
4. Flow F01 creates the 6 folders, saves the folder link, and emails Priya (thank you) and the APPG mailbox (new request) (section 14).

Phase 2 – APPG Review (13-16 Oct)

- **13-Oct:** David opens the APPG Queue → TV-2026-00015 → **Start Review**, with himself as APPG Owner. Status → Under APPG Review; Priya gets a "review started" email.
- **13-Oct:** David sees the wafer map is missing. He writes "Please attach the wafer map for the 25 wafers." and clicks **Request More Info**. Status → More Info Required; Priya's email shows his comment.

- **15-Oct:** Priya uploads Wafer_Map.xlsx, writes "Wafer map attached." and clicks **Resubmit**. Status → Under APPG Review; David is told. (Had Priya not answered: reminder on day 3, escalation to her manager on day 5, auto-cancel on day 30 – flow F05.)
- **16-Oct:** David is satisfied and clicks **Move to Internal Alignment**.

Phase 3 – Internal Alignment (17–31 Oct)

1st alignment – David asks 3 institutions for quotes and enters the options:

Option	Institution	Capability	Quote (USD)	Weeks	Constraint
A	Stanford Nanofab Lab	5	39,500	10	Lab free from Nov
B	Fraunhofer IPMS	4	35,000	16	Shipping to Germany
C	Internal Lab Fremont	3	22,000	20	Tool booked till Jan

The quotes go into 03-Feasibility & Quotes. Priya gets an alignment update email.

2nd alignment – a meeting with Priya and program manager Tom. Option A is chosen because it is the fastest. David clicks **Select this option** → **Confirm Option** (status Internal Alignment 2), adds Tom as a stakeholder (flow F10), and clicks **Send for Funding Approval**.

Why A and not the cheapest (C)? C is USD 17,500 cheaper but takes 20 weeks and its tool is booked until January. Capability (5 of 5) and speed won here.

Phase 4 – Funding Approval (2–3 Nov)

Flow F04 runs:

1. Cost center CC-4100 → owner **Maria Lopez**.
2. USD 39,500 is below the 50,000 limit → only one approval level.
3. Maria is added to the request's access team, so she can open it.
4. A Funding Approval row is created (Pending), and the approval appears in Maria's Outlook and Teams.

3-Nov: Maria clicks **Approve** with the comment "Approved from Q4 budget." The approval row becomes Approved; the request becomes Funding Approved with Approved Amount 39,500; Priya and David get the "funding approved" email.

(Had Maria rejected with "No budget this quarter": status Funding Rejected, Priya gets the reason, and the request ends.)

Phase 5 – Execution (10-Nov-2026 to 28-Feb-2027)

David clicks **Start Execution** → In Execution. He adds 4 milestones:

#	Milestone	Planned	Actual	%
1	Wafer preparation	10–24 Nov	12–26 Nov	100
2	Etch runs (3 recipes)	25 Nov – 15 Dec	27 Nov – 20 Dec	100
3	CD-SEM measurement	16 Dec – 20 Jan	21 Dec – 25 Jan	100
4	Final report	21 Jan – 15 Feb	26 Jan – 20 Feb	100

- **5-Dec:** issue "Lab tool down for maintenance", severity High → exception email to David and Priya (flow F02b). Action: move the run to a second tool. Resolved on 9-Dec.
- **Costs:** lab fee 18,000 (Nov) + lab fee 12,000 (Dec) + materials 5,300 + shipping 2,500 = **37,800** → 96% of the 39,500 approved.
- Expected completion is updated to 28-Feb; status updates are emailed to Priya and Tom.
- **28-Feb:** David clicks **Complete Execution** → Execution Completed.

Phase 6 – Closure (5-Mar-2027)

David uploads Final_Report.pdf to 06-Final Report & Closure and fills:

Field	Value
Final Outcome	Recipe B improved uniformity to 1.6% (target < 2%)
Lessons Learned	Book external lab 6 weeks early; keep backup tool option.
Reuse Recommendation	Reuse with changes

He clicks **Close Request** → status Closed, Closed On = 5-Mar-2027. Flow F07 builds the summary PDF TV-2026-00015_Summary_20270305.pdf in SharePoint and emails Priya, David, Maria and Tom: "TV-2026-00015 completed – final report available", with both links.

What management sees in Power BI

Measure	Value for TV-2026-00015
Cycle time	12-Oct-2026 → 5-Mar-2027 = 144 days
Cost	Approved 39,500; actual 37,800; variance +1,700 (under budget)
Bottleneck	Most time in In Execution (expected); about 2 days lost in More Info Required
Institution	Stanford Nanofab: 1 request, within budget

The full audit trail (Status History)

When	From → To	By
12-Oct 10:05	Draft → Submitted	Priya
13-Oct 09:12	Submitted → Under APPG Review	David
13-Oct 09:40	Under APPG Review → More Info Required	David
15-Oct 14:20	More Info Required → Under APPG Review	Priya
16-Oct 11:00	Under APPG Review → Internal Alignment 1	David
31-Oct 16:30	Internal Alignment 1 → Internal Alignment 2	David
02-Nov 09:00	Internal Alignment 2 → Pending Funding Approval	David
03-Nov 15:45	Pending Funding Approval → Funding Approved	Maria (via flow)
10-Nov 08:30	Funding Approved → In Execution	David
28-Feb 17:00	In Execution → Execution Completed	David
05-Mar 10:10	Execution Completed → Closed	David

That is 11 moves, matching the "11 rows" in section 6.

Two things in the story to read carefully

- **The Required By date was missed.** The guide says Option A was chosen because "only it meets the 15-Dec date", but 10 weeks from November ends in January, and the work finished on 28-Feb. So the request was late: the app shows Required By in red from 16-

Dec, and On-Time Completion % counts it as late. Read the reason as "it was the fastest option".

- **The sample CSV is an earlier moment of the same request.** In tv_requests.csv it is still In Execution, with 2 cost entries (18,000 + 5,300 = 23,300) and milestones at 100, 60, 0 and 0%. The story above is the end state.

21. The sample data – 10 CSV files explained

The 10 CSV files are fake test data for DEV and TEST only – never load them into PROD. They match the examples in the guide, and each request row was chosen to test one part of the process. Load them in the order of section 8.

Rules from the sample-data README

- **People are given by email** (for example maria.lopez@lam.com). They must already exist as users in the environment (they come from Entra ID). Replace them with your real test users before importing.
- **TVIDs are given** (TV-2026-00015 and so on) so that the examples line up. In real use the TVID is an autonumber. On import, map the TVID column to the autonumber column (this is allowed), otherwise Dataverse makes new numbers and the child files, which point to "TV-2026-00015", no longer match.
- **Choice values must use your exact labels**, for example "More Info Required", not "More info required".
- **Expected Power BI numbers** with this data: Total 8, Open 5, Closed 1, Rejected/Cancelled 2, Approved budget 82,500.

Files 1–5: master data and settings

1. departments.csv → Department (4 rows)

Name	Code	Department Head	Active
Process Engineering	PE	ravi.kumar@lam.com	Yes
Metrology	MET	susan.lee@lam.com	Yes
Deposition R&D	DEP	kevin.brown@lam.com	Yes
Product Engineering	PRD	anna.white@lam.com	Yes

2. locations.csv → Location (4 rows): Fremont, CA (USA) · Tualatin, OR (USA) · Bangalore (India) · Villach (Austria). All active.

3. research_institutions.csv → Research Institution (4 rows)

Name	Type	Contact	Capabilities
Stanford Nanofab Lab	University	Dr. Alan Green, agreen@stanford.example	Plasma etch, SEM, TEM
Fraunhofer IPMS	Research Lab	Dr. Eva Schmidt, eva.schmidt@ipms.example	Etch, deposition, metrology
Internal Lab Fremont	Internal Lab	Mark Davis, mark.davis@lam.com	Etch, CD-SEM
IMEC	Research Lab	Dr. Pieter Claes, pieter.claes@imec.example	Advanced nodes, EUV

The ".example" email domains are fake on purpose, so no test email reaches a real outside person.

4. cost_centers.csv → Cost Center (3 rows)

Name	Code	Owner	Owner's Manager	Department
CC-4100 Etch R&D	4100	maria.lopez@lam.com	james.wilson@lam.com	Process Engineering
CC-4200 Metrology	4200	susan.lee@lam.com	james.wilson@lam.com	Metrology
CC-4300 Deposition R&D	4300	kevin.brown@lam.com	james.wilson@lam.com	Deposition R&D

5. app_settings.csv → App Setting (5 rows): ReminderAfterDays = 3, EscalationAfterDays = 5, AutoCancelAfterDays = 30, HighValueFundingLimit = 50000, RetentionYears = 7 (explained in section 7).

File 6: email_templates.csv → Email Template (15 rows)

Each template has a code, a subject, an HTML body and Is Active = Yes. This table shows when each one is sent:

Template code	Sent when	Sent by
REQ_SUBMITTED	A request is submitted (to the requester)	F01

Template code	Sent when	Sent by
NEW_REQUEST_APPG	A request is submitted (to the APPG team)	F01
REVIEW_STARTED	Status becomes Under APPG Review	F02
MORE_INFO	Status becomes More Info Required (shows the comment)	F02
REJECTED	Status becomes Rejected (shows the reason)	F02
ALIGNMENT_UPDATE	Internal Alignment 1 or 2	F02
FUNDING_APPROVED	Funding Approved	F02 (after F04)
FUNDING_REJECTED	Funding Rejected	F02 (after F04)
STATUS_UPDATE	In Execution, On Hold, Execution Completed	F02
EXCEPTION_RAISED	A High severity issue is added	F02b
REMINDER	An action has waited 3+ days (uses {Days})	F05
ESCALATION	An action has waited 5+ days (uses {Days}, {ActionOwner})	F05
CANCELLED	Status becomes Cancelled	F02
COMPLETED	Status becomes Closed (uses {ReportLink}, {PdfLink})	F07
NEXT_OCCURRENCE	A Recurring request is closed	F07

Missing: STAKEHOLDER_ADDED, which flow F10 needs. Add it as a 16th row.

File 7: tv_requests.csv → TV Request (8 rows, 23 columns)

Each row tests a different part of the process:

TVID	Title	Type	Requester	Status	What it tests
TV-2026-00001	Quarterly contamination check	Recurring (Quarterly)	John (Metrology)	Closed (approved 15,000; closed 25-Jun-2026)	A finished request; cycle time; recurring
TV-2026-00003	Chamber material stress test	One-time	Priya (PE)	In Execution (approved 28,000)	Execution; the "parent" of a Repeat

TVID	Title	Type	Requester	Status	What it tests
TV-2026-00009	Low-k film adhesion test	New	Kevin (Deposition R&D, Tualatin)	Cancelled – "Auto-cancelled: no response for 30 days"	Auto-cancel
TV-2026-00011	Plasma etch uniformity (duplicate)	New	John (Metrology)	Rejected – "Duplicate of TV-2026-00015 scope"	Rejection with a reason
TV-2026-00012	EUV resist outgassing study	New	Anna (Product Engineering, Villach)	Pending Funding Approval (62,000)	Two-level approval (above 50,000)
TV-2026-00015	Plasma etch uniformity test on 300mm wafers	New	Priya (PE, Fremont)	In Execution (approved 39,500)	The main story (section 20)
TV-2026-00016	Repeat chamber stress test with new liner	Repeat of TV-2026-00003	Priya (PE)	More Info Required	Repeat request; escalation
TV-2026-00018	Metrology tool correlation study	One-time	John (Metrology, Bangalore)	Submitted, no APPG owner yet	Reminder to the APPG team

The 23 columns are: TVID, Title, Type, Previous Request, Frequency, Requester (email), Department, Location, Cost Center, Priority, Required By, Estimated Budget, Status, Phase, APPG Owner (email), Approved Amount, Submitted On, Status Changed On, Closed On, Business Justification, Technical Description, Expected Outcome, and Rejection/Cancellation Reason.

A business question hidden in this data: Kevin Brown requested TV-2026-00009 on CC-4300, and he is also the owner of CC-4300. Had it reached funding, Kevin would have approved his own request. Ask LAM whether the system should block self-approval (for example, send it to the Owner's Manager instead).

Files 8-10: child data

8. feasibility_options.csv → Feasibility Option (5 rows)

Option	Request	Institution	Score	Cost	Weeks	Constraint	Recommendation	Selected
Option A – Stanford Nanofab	TV-2026-00015	Stanford Nanofab Lab	5	39,500	10	Lab free from Nov	Recommended	Yes
Option B – Fraunhofer	TV-2026-00015	Fraunhofer IPMS	4	35,000	16	Shipping to Germany	Alternative	No

Option	Request	Institution	Score	Cost	Weeks	Constraint	Recommendation	Selected
Option C - Internal Lab	TV-2026-00015	Internal Lab Fremont	3	22,000	20	Tool booked till Jan	Not Recommended	No
Option A - IMEC	TV-2026-00012	IMEC	5	62,000	12	EUV slot in Dec	Recommended	Yes
Option A - Internal Lab	TV-2026-00003	Internal Lab Fremont	4	28,000	8	None	Recommended	Yes

9. milestones.csv → Milestone (6 rows)

Milestone	Request	Seq	Planned	Actual start / end	%	Status
Wafer preparation	TV-2026-00015	1	10-Nov - 24-Nov-2026	12-Nov / 26-Nov	100	Completed
Etch runs (3 recipes)	TV-2026-00015	2	25-Nov - 15-Dec-2026	27-Nov / -	60	In Progress
CD-SEM measurement	TV-2026-00015	3	16-Dec-2026 - 20-Jan-2027	-	0	Not Started
Final report	TV-2026-00015	4	21-Jan - 15-Feb-2027	-	0	Not Started
Liner installation	TV-2026-00003	1	20-Aug - 5-Sep-2026	21-Aug / 6-Sep	100	Completed
Thermal cycling	TV-2026-00003	2	6-Sep - 31-Oct-2026	7-Sep / -	70	Delayed

Overall progress from this data: TV-2026-00015 = $(100 + 60 + 0 + 0) \div 4 = 40\%$; TV-2026-00003 = $(100 + 70) \div 2 = 85\%$.

10. cost_entries.csv → Cost Entry (5 rows)

Description	Request	Category	Amount	Date	Invoice
Lab usage fee - Nov	TV-2026-00015	Lab Fee	18,000	2026-11-30	INV-88213
Wafers and consumables	TV-2026-00015	Material	5,300	2026-11-15	INV-88190
Lab usage fee - Aug/Sep	TV-2026-00003	Lab Fee	14,000	2026-09-30	INV-87002
Liner material	TV-2026-00003	Material	6,000	2026-08-18	INV-86950
Quarterly check fee	TV-2026-00001	Lab Fee	14,200	2026-06-20	INV-85010

Utilization per request from this data (my calculation): TV-2026-00015 = $23,300 \div 39,500 = 59\%$; TV-2026-00003 = $20,000 \div 28,000 = 71\%$; TV-2026-00001 = $14,200 \div 15,000 = 95\%$.
Use these to check the Cost Tracking page in Power BI.

22. Corrections, gaps and open questions – all in one place

The guide is a strong starting point, but while explaining it I found 6 technical errors to fix, 12 design gaps to fill, and 11 points to confirm with LAM. None of them changes the overall plan; most are small changes. They are collected here so you can work through them as a checklist.

A. Fix before you build (technical errors)

#	In the guide	Problem	What to do	Details
A1	Submit button uses <code>Exit()</code> when the save fails	In Power Apps, <code>Exit()</code> closes the whole app	Use <code>IfError(...)</code> that returns <code>Blank()</code> , then <code>If(!IsBlank(varNew), ...)</code>	Section 12
A2	<code>Navigate()</code> inside <code>App.OnStart</code> for the email link	Not allowed in current Power Apps	<code>App.StartScreen</code> + load the request in <code>scrRequestDetail.OnVisible</code>	Section 11
A3	<code>AddUserToRecordTeam</code> as an "unbound action"	In the flow designer it is normally a bound action on Users	"Perform a bound action": Table = Users, Row = the person, plus Record and TeamTemplate	Sections 10, 15
A4	Actual Total Cost as a rollup, shown at once in the app	Rollups recalculate on a schedule, not instantly	Show <code>Sum(Filter('Cost Entries', ...))</code> in the app	Sections 8, 12
A5	Business rule "Required By must be after today"	Business rules have no built-in "today" value	Keep this check in the Canvas app	Section 8

#	In the guide	Problem	What to do	Details
A6	SharePoint versioning settings	"Require check out" on would hide files uploaded by flows	Keep "Require check out" = No	Section 9

B. Add to the design (gaps inside the guide)

#	Gap	What to do	Details
B1	The Requester role cannot create Status History rows, but Submit and Resubmit write them	Give Create + Append (User level), or let a flow write history	Section 10
B2	Optional column security on tvr_status would also block the requester's own Submit, Resubmit and Cancel	If you lock it, run those 3 actions through a flow	Section 10
B3	Requesters may not see child rows that APPG created	Test it; if needed set Reparent = Cascade All	Section 8
B4	Submit has no Recurring-frequency check (TC-06)	Add the check, or make the Frequency card required for Recurring	Section 17
B5	"Comment mandatory" for Request More Info is not written out (TC-10)	Copy the Reject pattern	Section 17
B6	F02 does not tell APPG about a resubmit (TC-11)	Add a case based on the newest Status History row	Section 17
B7	F02 has no step to find the "stakeholders"	Read the access team members, or keep a Stakeholder Emails column	Section 14
B8	{Days}, {ActionOwner}, {ReportLink}, {PdfLink} are not replaced by the email block	Add replace steps in F05 and F07	Section 15
B9	Template STAKEHOLDER_ADDED is missing	Add a 16th template row	Sections 15, 21

#	Gap	What to do	Details
B10	F09 gets DocType as text, but Document Type is a choice number; Phase is not an input	Convert text to number; read Phase from the request first	Section 15
B11	varDraft and varCopyFrom are not cleared when a new request starts	Clear them in btnNewRequest	Section 12
B12	Status History "Changed By" is left empty by the app	Add 'Changed By' : CurrentUserRecord	Section 12

C. Confirm with LAM (add these to the Week 1 questions)

#	Question	Why	Details
C1	Working days or calendar days for reminders? (G5 says working, F05 uses calendar)	Day counting	Sections 3, 15
C2	A "Create next occurrence" button, or only the NEXT_OCCURRENCE email?	Recurring requests	Section 3
C3	May APPG reject in Internal Alignment 1? (buttons allow it, the status table does not)	Status rules	Section 7
C4	Count days by calendar date or by full 24 hours?	The F05 sample table and formula disagree	Section 15
C5	One escalation, or one every weekday after day 5?	Email volume	Section 15
C6	Should a cost center owner be blocked from approving their own request?	Kevin Brown in the sample data	Section 21
C7	Keep the requester's first estimate, or overwrite it with the selected quote?	Estimated Budget is overwritten	Section 12
C8	Should the TVID counter restart at 00001 each year?	The autonumber does not restart	Section 7
C9	Should the Overdue view also exclude Cancelled and Rejected requests?	Cancelled requests show as overdue	Section 13

#	Question	Why	Details
C10	Simple SharePoint permissions (everyone reads) or strict (only own folders)?	Security and effort	Section 9
C11	Who are the stakeholders, and who adds them?	Access team and emails	Sections 10, 14

Two small slips in the guide's own examples, for your information: the security test calls TV-2026-00016 "John's" but it is Priya's (use TV-2026-00011 or TV-2026-00018), and the story says Option A "meets the 15-Dec date" although the work finished on 28-Feb (sections 10 and 20).

Your first steps

1. **This week:** send the 21 questions from section 3, plus table C above, to your manager. Ask for answers by the end of Week 1.
2. **In parallel:** confirm licences and get DEV, TEST and PROD (the checklist in section 5).
3. **In DEV:** create the publisher, the solution and the 4 global choices; build the tables in the order of sections 7 and 8; load the sample data.
4. **While building:** fix table A as you reach each item, and add table B into your Solution Blueprint.
5. **Sprint 1:** build stories US-01 to US-10 (section 19) and use test cases TC-01 to TC-12 and TC-27 (section 17) as your "done" check.